ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN



Môn học: **Công nghệ Mạng và Truyền thông hiện đại**

Mã lớp: **IT2003.11.CH**

Giảng viên: **PGS TS. Lê Trung Quân**

Học kì I - Năm học 2026-2027

Tên đề tài:
Cilium and eBPF-based Networking for Kubernetes: A Modern Alternative to Traditional iptables Networking

---

Mã nhóm SV thực hiện: Nhóm 12

Thông tin thành viên nhóm:

MSSV	Họ và tên
250202052	Nguyễn Hoài Phương
250202060	Nguyễn Long Vũ
26848033	Ngô Vũ Minh Đạt

Ngày 31 tháng 10 năm 2026

Mục lục

1 Mục tiêu đồ án **3**

2 Tổng quan nghiên cứu **3**

3 Cơ sở lý thuyết **4**
3.1 Mô hình mạng Kubernetes . . . 4
3.2 Các primitive mạng của nhân Linux và CNI . . . 4
3.3 eBPF . . . 6
3.4 Đường đi của gói tin theo từng lớp . . . 7
3.4.1 Intra-node: hai Pod trên cùng một host . . . 7
3.4.2 Inter-node: định tuyến trực tiếp . . . 7
3.5 Cân bằng tải Service: kube-proxy iptables và Cilium eBPF . . . 8
3.6 Thực thi NetworkPolicy: iptables và eBPF . . . 9

4 Phương pháp Triển Khai **10**
4.1 Hạ tầng và topology mạng . . . 10

5 Kết Quả Thực Nghiệm và Đánh Giá **12**
5.1 E1 – Hiệu năng Service theo quy mô . . . 12
5.2 E2 – Biểu diễn và thực thi NetworkPolicy trên Cilium và Calico . . . 15
5.2.1 Cilium: cấu hình chính sách mạng . . . 16
5.2.2 Cilium: giới hạn kết nối giữa các dịch vụ . . . 16
5.2.3 Cilium: chặn kết nối reverse shell từ DVWA . . . 17
5.2.4 Cilium: kiểm soát HTTP API của DVWA ở lớp 7 . . . 19
5.2.5 Tổng hợp và đánh giá kết quả Cilium . . . 21
5.2.6 Calico: giới hạn kết nối giữa các dịch vụ . . . 21
5.2.7 Calico: chặn kết nối reverse shell từ DVWA . . . 22
5.2.8 Tổng hợp và đánh giá kết quả Calico . . . 23
5.3 E3 – Giám sát hành vi truy cập tệp và kết nối mạng bằng Tetragon . . . 24
5.3.1 Cấu hình và cách theo dõi . . . 24
5.3.2 Theo dõi tiến trình từ Command Injection . . . 24
5.3.3 Theo dõi kết nối TCP . . . 25
5.3.4 Theo dõi đọc tệp ServiceAccount và gọi Kubernetes API . . . 25
5.3.5 Nhận xét . . . 28

6 Kết Luận **28**

Danh mục hình ảnh

1 Primitive mạng của Pod: netns, veth và CNI . . . 5
2 Netfilter và conntrack: chỉ gói tin đầu tiên duyệt chuỗi luật . . . 6
3 Vòng đời chương trình eBPF . . . 6
4 Đường đi gói tin giữa hai Pod trên cùng một node . . . 7
5 Định tuyến trực tiếp giữa hai node . . . 8
6 Tra cứu Service: chuỗi iptables và BPF hash map . . . 9
7 Ba bước thực thi NetworkPolicy với iptables và eBPF . . . 9
8 Topology hạ tầng homelab . . . 11
9 Quy trình tạo số liệu E1 . . . 13
10 Kết quả E1 . . . 14
11 Danh sách Pod của Online Boutique và địa chỉ IP dùng trong phép thử: frontend `10.244.1.60`, Redis `10.244.2.95` và Payment `10.244.2.27`. . . . 16
12 Kiểm tra kết nối từ frontend tới Redis và Payment: hai kết nối TCP thành công ở lần kiểm tra ban đầu (trên); các gói SYN bị từ chối với lý do `Policy denied` khi kiểm tra thực thi chính sách (dưới). . . . 17
13 Policy `dvwa-restrict-egress`: giới hạn egress của DVWA, cho phép DNS trên UDP/TCP 53 và Kubernetes API trên TCP 443/6443. . . . 18

14 Kết quả kiểm thử reverse shell: máy lắng nghe nhận được shell từ DVWA trong phép thử đối chứng (trên); sau khi giới hạn egress, Hubble ghi nhận kết nối tới `10.8.0.10:1234` bị từ chối (dưới). . . . 19
15 Policy `dvwa-l7-api-policy` kiểm soát ingress TCP 80 theo phương thức và đường dẫn HTTP. Các đường dẫn GET được phép bao gồm API v2. . . . 20
16 Yêu cầu GET tới API v1 của DVWA bị chặn: trình duyệt trả về `Access denied`, đồng thời Hubble ghi nhận verdict `http-request DROPPED`. . . . 20
17 Cụm Calico: kết quả `nc`/`dig` từ frontend (trái) và luật iptables tương ứng, kèm tên chính sách (phải). . . . 22
18 Cụm Calico: máy nghe ngoài cụm không ghi nhận kết nối (trái); `tcpdump` trên node chứa Pod DVWA ghi nhận 4 lần truyền lại SYN không có `SYN-ACK` (phải). . . . 23
19 Tetragon ghi lại các tiến trình và mã thoát trong Pod DVWA. Dòng lệnh của shell cho thấy lệnh ping và phần lệnh echo được chạy qua Command Injection. . . . 24
20 Policy theo dõi hàm `tcp_connect`, với tham số kiểu `sock` và hành động `Post`. . . . 25
21 Sự kiện `connect` của Apache tới `127.0.0.1:3306`, cùng các sự kiện tiến trình trong Pod DVWA. 25
22 Policy theo dõi đọc tệp qua hàm `security_file_permission`. Rule lọc các đuôi `/token`, `/ca.crt`, `/namespace` và mặt nạ quyền đọc có giá trị 4. . . . 26
23 Tetragon ghi nhận Apache đọc token, tệp CA của ServiceAccount và gọi `tcp_connect` tới `192.168.40.160:6443`. Các sự kiện đều thuộc Pod DVWA. . . . 27
24 Kết quả chương trình kiểm thử trong DVWA: truy cập API lấy phiên bản và thông tin ServiceAccount. Các dòng kiểm tra quyền cần được kiểm tra lại cách xử lý phản hồi. . . . 27

Danh mục bảng

1 So sánh cơ chế thực thi NetworkPolicy giữa hai datapath (trong ngoặc: hiện thực được dùng trong đồ án). . . . 10
2 Danh sách máy ảo của môi trường thực nghiệm. . . . 11
3 So sánh cấu hình hai cụm Kubernetes. . . . 12
4 Tổng hợp kết quả E1 (trung bình ± độ lệch chuẩn của 3 lần chạy). Kích thước bảng: số luật nat (iptables) và số dòng LB map (Cilium) trên node client. . . . 14
5 Kết quả kiểm thử chính sách mạng trên cụm Cilium. . . . 21
6 Kết quả kiểm thử chính sách mạng trên cụm Calico. . . . 23
7 Kết quả giám sát bằng Tetragon trong E3. . . . 28
8 Tổng hợp sự khác biệt giữa Netfilter/iptables và Cilium eBPF 29

Tiêu đề đồ án: Cilium and eBPF-based Networking for Kubernetes: A Modern Alternative to Traditional iptables Networking

Tóm tắt nội dung

Trong Kubernetes, lớp mạng (CNI) và cơ chế cân bằng tải Service quyết định trực tiếp độ trễ, khả năng mở rộng, khả năng thực thi chính sách bảo mật và khả năng quan sát của toàn bộ cụm. Trong nhiều năm, giải pháp phổ biến là kết hợp kube-proxy và một CNI dựa trên Netfilter/iptables như Calico. Mô hình này thực hiện DNAT và lọc gói tin bằng các chuỗi luật được duyệt tuần tự ($O(n)$), cập nhật luật theo kiểu nạp lại cả bảng, gắn chính sách mạng với địa chỉ IP tạm thời của Pod và chỉ cung cấp bộ đếm gói tin khi gói bị loại bỏ. eBPF cho phép nạp chương trình đã được kiểm chứng vào nhân Linux tại các điểm móc như TC (datapath mạng) hay kprobe (theo dõi hàm của nhân). Dựa trên eBPF, Cilium thay thế kube-proxy bằng bảng băm trong nhân ($O(1)$), thực thi chính sách theo định danh bảo mật (security identity) thay vì địa chỉ IP, cung cấp khả năng quan sát luồng mạng theo thời gian thực qua Hubble, và qua Tetragon quan sát hành vi tiến trình và truy cập tệp ngay trong nhân.

Đồ án tái hiện và đánh giá thực nghiệm sự khác biệt giữa hai kiến trúc trên hai cụm Kubernetes ba node có cấu hình phần cứng giống hệt nhau, triển khai song song trên Proxmox bằng Terraform, Ansible và Kubespray, qua ba thí nghiệm: (E1) hiệu năng thiết lập kết nối khi số lượng Service tăng dần; (E2) biểu diễn và thực thi NetworkPolicy theo định danh để phân đoạn ứng dụng Online Boutique, kiểm chứng bằng Hubble và `cilium-dbg monitor`; (E3) giám sát hành vi thời gian chạy bằng Tetragon sau khi khai thác DVWA để đọc token ServiceAccount (chỉ giám sát, không chặn). Ở E1, khi số Service tăng từ 100 lên 10 000, độ trễ trung vị (p50) của iptables tăng tuyến tính từ 2.33 ms lên 4.93 ms (khoảng +0.26 ms mỗi 1 000 Service), trong khi Cilium giữ quanh 2.4 ms; p99 tại 10 000 Service là 9.93 ms so với 4.96 ms. Ở E2, datapath của Cilium ghi nhận các gói tin bị loại bỏ kèm lý do `Policy denied`. Ở E3, Tetragon ghi nhận các tiến trình phát sinh từ Command Injection trong Pod DVWA. Kết quả giúp đưa ra khuyến nghị lựa chọn datapath cho các cụm Kubernetes có quy mô và yêu cầu bảo mật khác nhau.

1 Mục tiêu đồ án

Mục tiêu của đồ án là:

- Tìm hiểu sâu mô hình mạng Kubernetes và hai cách hiện thực datapath: Netfilter/iptables (Calico + kube-proxy) và eBPF (Cilium).
- Triển khai thực tế hạ tầng và hai cụm Kubernetes tương đương để có môi trường so sánh công bằng.
- Thực hiện và đánh giá các thí nghiệm so sánh về hiệu năng Service, thực thi chính sách mạng và khả năng quan sát.
- Rút ra nhận xét và khuyến nghị lựa chọn giải pháp mạng phù hợp cho cụm Kubernetes.

2 Tổng quan nghiên cứu

Bối cảnh và vấn đề. Kubernetes định nghĩa một mô hình mạng phẳng: mỗi Pod có một địa chỉ IP riêng và có thể giao tiếp với mọi Pod khác mà không cần NAT [1, 2]. Việc hiện thực hoá mô hình này được giao cho hai thành phần: plugin CNI (cấp IP, nối mạng cho Pod, định tuyến giữa các node, thực thi NetworkPolicy) và kube-proxy (hiện thực địa chỉ ảo ClusterIP của Service). Trong phần lớn lịch sử của Kubernetes, cả hai thành phần đều dựa trên Netfilter/iptables, một framework lọc gói tin đã ổn định hơn hai thập kỷ nhưng được thiết kế cho các tập luật tường lửa tương đối tĩnh.

Ở quy mô cloud-native, với hàng nghìn Service, Pod được tạo và huỷ liên tục và lượng kết nối ngắn lớn, thiết kế này bộc lộ các giới hạn đã được ghi nhận [3]:

- **Độ phức tạp tuyến tính:** gói tin đầu tiên của mỗi kết nối phải được so khớp tuần tự với chuỗi luật Service, nên chi phí tăng theo số Service.

- **Cập nhật nguyên khối:** thay đổi một Service kéo theo việc ghi lại (một phần hoặc toàn bộ) bảng luật qua `iptables-restore` trong user space.
- **Tranh chấp conntrack:** bảng conntrack dùng chung của nhân trở thành điểm nghẽn khi tốc độ tạo kết nối mới cao.
- **Chính sách gắn với IP:** NetworkPolicy được chuyển thành tập địa chỉ IP, mà IP của Pod là tạm thời, nên mỗi lần Pod thay đổi lại phải đồng bộ trên mọi node.
- **Khả năng quan sát kém:** khi gói tin bị loại bỏ, nhân chỉ tăng bộ đếm của luật, không ghi lại Pod nguồn/đích hay chính sách nào gây ra.

Các giải pháp hiện có. Cộng đồng đã đề xuất nhiều hướng giải quyết: (i) chế độ IPVS của kube-proxy dùng bảng băm cho cân bằng tải nhưng vẫn phụ thuộc iptables cho các chức năng khác; (ii) chế độ nftables của kube-proxy dùng cấu trúc map/set thay cho chuỗi tuyến tính; (iii) Calico eBPF dataplane; (iv) Cilium, CNI được thiết kế xoay quanh eBPF ngay từ đầu, có khả năng thay thế hoàn toàn kube-proxy, thực thi chính sách theo định danh và tích hợp Hubble/Tetragon cho quan sát và bảo mật thời gian chạy [3, 4].

Khoảng trống và câu hỏi nghiên cứu. Đồ án đặt ra câu hỏi: *trên một cụm nhỏ, phần cứng phổ thông, triển khai bằng công cụ chuẩn (Kubespray), các khác biệt về mặt lý thuyết giữa hai datapath có thể quan sát và kiểm chứng được đến mức nào?* Cụ thể:

- **RQ1 (E1):** Độ trễ thiết lập kết nối tới Service thay đổi thế nào khi số Service tăng, với iptables và với Cilium eBPF?
- **RQ2 (E2):** NetworkPolicy được biểu diễn trong nhân như thế nào ở mỗi giải pháp, và việc thêm Pod mới ảnh hưởng thế nào đến trạng thái chính sách?
- **RQ3 (E3):** Người vận hành cần những bước và thông tin gì để xác định nguyên nhân một kết nối bị chặn?

3 Cơ sở lý thuyết

Phần này trình bày các khái niệm nền tảng theo cách tiếp cận phân lớp của Strong and Lancey [2], rồi đi vào hai khía cạnh được đánh giá trong đồ án: cân bằng tải Service và thực thi chính sách.

3.1 Mô hình mạng Kubernetes

Kubernetes không tự hiện thực mạng mà đặt ra một tập ràng buộc mà mọi giải pháp mạng phải thoả mãn [1, 2]:

1. Mọi Pod có thể giao tiếp với mọi Pod khác trên bất kỳ node nào mà **không cần NAT**.
2. Các tác tử trên node (ví dụ kubelet) có thể giao tiếp với mọi Pod trên node đó.
3. Địa chỉ IP mà Pod tự nhìn thấy trùng với địa chỉ IP mà các thành phần khác dùng để kết nối tới nó.

Hệ quả là cụm có một **không gian địa chỉ phẳng**: mỗi Pod có một IP duy nhất trong dải Pod CIDR, và mỗi node thường được cấp một dải con. Tuy nhiên, IP của Pod là tạm thời, thay đổi mỗi khi Pod được tạo lại. Để có điểm truy cập ổn định, Kubernetes định nghĩa đối tượng **Service**: một địa chỉ IP ảo (ClusterIP) cùng cổng, ánh xạ tới tập các Pod backend (Endpoints/EndpointSlices) được chọn qua label selector. ClusterIP không gắn với bất kỳ giao diện mạng nào; nó chỉ tồn tại dưới dạng luật dịch địa chỉ (DNAT) trên mỗi node. Thành phần cài các luật này theo truyền thống là **kube-proxy**, chạy dưới dạng DaemonSet và theo dõi API server để đồng bộ Service/Endpoints vào nhân Linux.

3.2 Các primitive mạng của nhân Linux và CNI

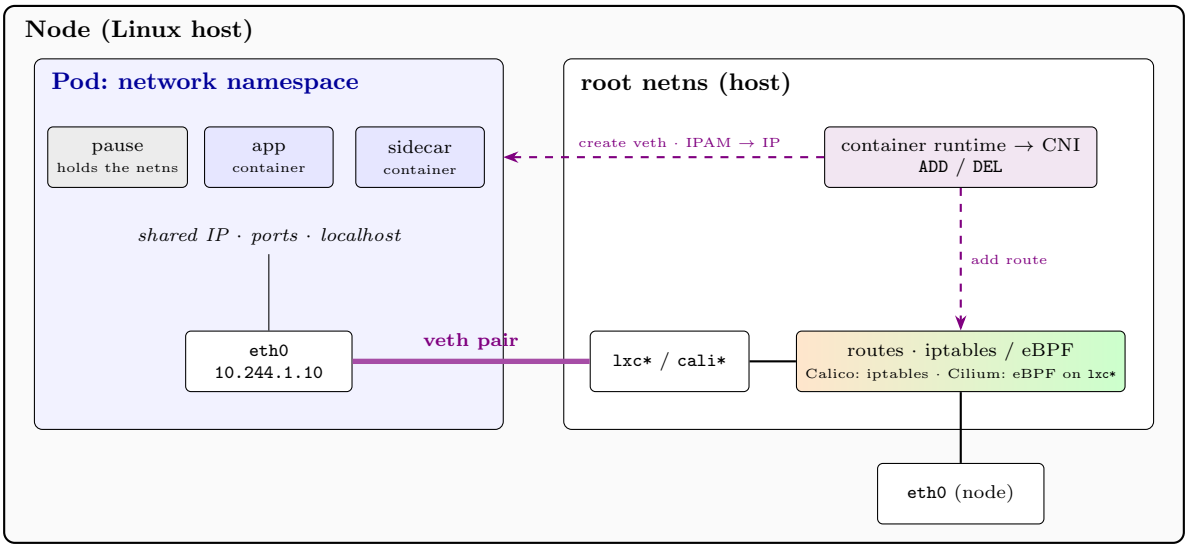
Network namespace (netns). Network namespace là cơ chế của nhân Linux cho phép cô lập toàn bộ ngăn xếp mạng: giao diện mạng, bảng định tuyến, bảng ARP, luật iptables và bảng conntrack. Mỗi Pod chạy trong một netns riêng; các container trong cùng Pod dùng chung netns đó, nên dùng chung IP và giao tiếp với nhau qua `localhost` [2]. Netns được giữ bởi container hạ tầng *pause*, nên khi một container ứng dụng khởi động lại, IP của Pod vẫn được giữ nguyên.

Cặp Ethernet ảo (veth). veth là cặp giao diện ảo hoạt động như một sợi cáp hai đầu: gói tin đi vào đầu này sẽ đi ra ở đầu kia. Một đầu được đặt trong netns của Pod (thường đặt tên `eth0`), đầu còn lại nằm trong netns gốc của host (ví dụ `cali*` với Calico, `lxc*` với Cilium). Đây là đường nối duy nhất giữa Pod và phần còn lại của mạng.

Container Network Interface (CNI). CNI là đặc tả giao tiếp giữa container runtime (containerd, CRI-O) và plugin mạng. Runtime tạo netns rồi gọi binary của plugin, truyền cấu hình qua `stdin` và các tham số qua biến môi trường. Hai thao tác chính là:

- **ADD**: tạo cặp veth, đưa một đầu vào netns của Pod, gọi plugin IPAM để cấp địa chỉ IP, cấu hình IP và route mặc định trong Pod, rồi nối đầu còn lại vào mạng của host (bridge hoặc route).
- **DEL**: giải phóng IP và dọn dẹp giao diện khi Pod bị huỷ.

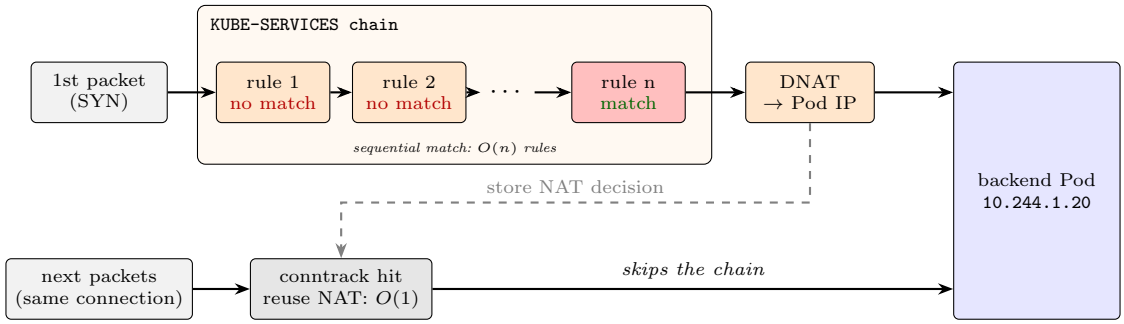
Với các CNI dựa trên eBPF như Cilium, binary CNI chỉ gửi yêu cầu tới **cilium-agent** trên node. Agent tạo endpoint, biên dịch/nạp chương trình eBPF vào các điểm móc của giao diện veth và cập nhật các BPF map tương ứng. Hình 1 tóm tắt quan hệ giữa netns, cặp veth và thao tác ADD của CNI.



Hình 1: Primitive mạng của một Pod: các container dùng chung network namespace (được giữ bởi container *pause*); cặp veth nối `eth0` của Pod với root netns; plugin CNI (thao tác `ADD`) tạo veth, cấp IP và thêm route.

Netfilter và iptables. Netfilter là framework trong nhân Linux cung cấp năm điểm móc trên đường đi của gói tin: `PREROUTING`, `INPUT`, `FORWARD`, `OUTPUT`, `POSTROUTING`. iptables là công cụ user space dùng để khai báo luật gắn vào các điểm móc này, tổ chức thành các bảng (`filter`, `nat`, `mangle`, `raw`) và chuỗi (chain). Mỗi chuỗi là một **danh sách luật được duyệt tuần tự**: gói tin được so khớp lần lượt từng luật cho đến khi gặp một target kết thúc (`ACCEPT`, `DROP`, `DNAT`, ...) hoặc nhảy sang chuỗi khác.

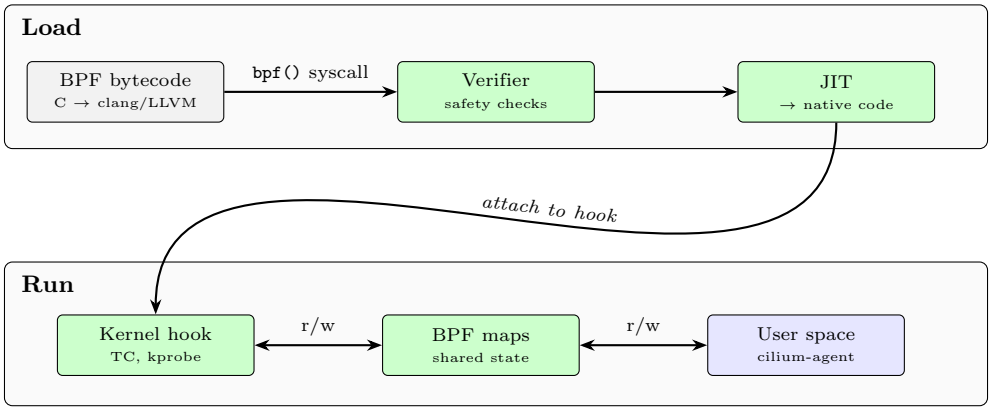
Đi kèm là **conntrack** (connection tracking), bảng trạng thái kết nối dùng chung của nhân. Với bảng `nat`, chỉ gói tin đầu tiên của một kết nối phải duyệt các chuỗi luật; quyết định NAT sau đó được lưu trong conntrack và áp dụng cho các gói tiếp theo. Vì vậy chi phí duyệt luật của iptables tập trung ở **giai đoạn thiết lập kết nối**, và tác động rõ nhất với các tải có nhiều kết nối ngắn. Đây là lý do thí nghiệm E1 đo kết nối mới cho mỗi yêu cầu (Hình 2).



Hình 2: Netfilter và conntrack: chỉ gói tin đầu tiên của một kết nối duyệt tuần tự chuỗi `KUBE-SERVICES` rồi DNAT; quyết định NAT được lưu vào conntrack và các gói tin sau dùng lại mà không duyệt chuỗi.

3.3 eBPF

Khái niệm. eBPF (extended Berkeley Packet Filter) là một máy ảo nằm trong nhân Linux, cho phép nạp và thực thi chương trình do người dùng viết tại các điểm móc của nhân mà không cần sửa mã nguồn nhân hay nạp kernel module. Quy trình gồm: (1) chương trình được viết bằng tập con của C và biên dịch ra bytecode eBPF bằng clang/LLVM; (2) nạp vào nhân qua system call `bpf()`; (3) **verifier** kiểm tra tính an toàn (không truy cập bộ nhớ ngoài phạm vi, vòng lặp có giới hạn, luôn kết thúc); (4) bytecode được **JIT** biên dịch sang mã máy gốc; (5) chương trình được gắn vào điểm móc và chạy mỗi khi sự kiện xảy ra (Hình 3).



Hình 3: Vòng đời chương trình eBPF: bytecode được nạp qua `bpf()`, kiểm tra bởi verifier, JIT sang mã máy rồi gắn vào điểm móc của nhân; khi chạy, chương trình đọc/ghi trạng thái trong BPF map, nơi user space (cilium-agent) cũng truy cập được.

Các điểm móc liên quan đến mạng.

- **TC** (Traffic Control) ingress/egress: chạy trên `sk_buff` tại mọi giao diện, kể cả veth. Cilium dùng TC làm điểm thực thi chính cho định tuyến, NAT và chính sách.
- **kprobe, tracepoint, LSM**: gắn vào hàm nhân, system call và các điểm kiểm tra bảo mật, được dùng cho quan sát và thực thi bảo mật thời gian chạy (Tetragon).

BPF map. Chương trình eBPF không giữ trạng thái giữa các lần chạy; trạng thái được lưu trong **BPF map**, các cấu trúc dữ liệu key-value nằm trong nhân, có thể được đọc/ghi đồng thời từ chương trình eBPF và từ user space. Các loại map dùng trong đồ án gồm hash và LRU hash (Service, conntrack, chính sách), LPM trie (tra cứu theo CIDR) và ring buffer/perf event array (đẩy sự kiện lên user space). Thao tác cập nhật một phần tử là nguyên tử, nên có thể thay đổi trạng thái datapath từng phần mà không cần nạp lại chương trình. Kích thước tối đa của map được cố định khi tạo, nên phải được định cỡ theo quy mô cụm (số Service, backend, mục conntrack, NAT) [5].

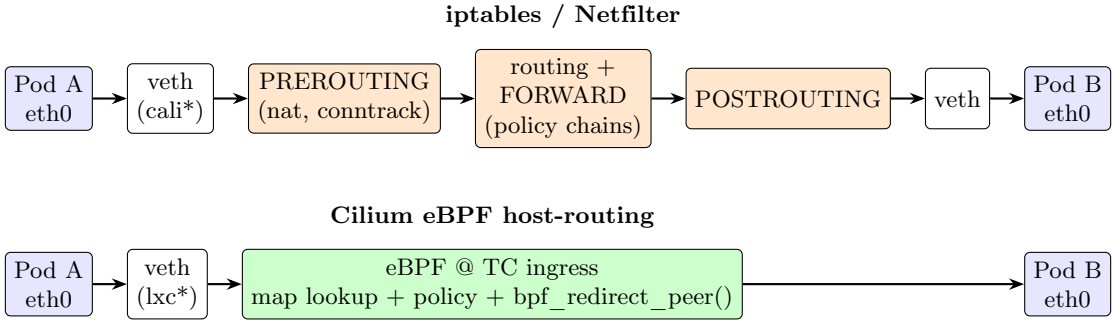
Cilium. Cilium là CNI xây dựng trên eBPF. Trên mỗi node, **cilium-agent** theo dõi API server, quản lý endpoint và nạp chương trình eBPF vào các giao diện (`lxc*` của Pod, giao diện vật lý, `cilium_host`, `cilium_vxlan`).

Trạng thái datapath được biểu diễn qua các map như `cilium_lb4_services_v2` và `cilium_lb4_backends_v3` (Service và backend), `cilium_ct4_global` (conntrack riêng của Cilium), `cilium_ipcache` (ánh xạ IP sang định danh bảo mật) và `cilium_policy_*` (chính sách của từng endpoint) [5].

3.4 Đường đi của gói tin theo từng lớp

3.4.1 Intra-node: hai Pod trên cùng một host

- **Mô hình truyền thống (bridge/iptables):** gói tin rời Pod A qua veth vào netns của host, đi qua bridge Linux (hoặc bảng định tuyến của host, như cách Calico dùng route /32 cho từng Pod), duyệt các chuỗi Netfilter (`PREROUTING`, `FORWARD`, `POSTROUTING`, gồm các chuỗi chính sách của CNI), rồi đi qua veth thứ hai vào netns của Pod B. Gói tin đi qua ngăn xếp mạng của host và Netfilter một lần đầy đủ, chưa kể ngăn xếp trong từng Pod.
- **Cilium eBPF host-routing:** chương trình eBPF tại TC ingress của veth phía host (`lxc*`) thực hiện tra cứu endpoint/route trong map, áp dụng chính sách, rồi dùng helper `bpf_redirect_peer()` chuyển gói tin thẳng vào giao diện của Pod đích. Gói tin bỏ qua bridge, bảng định tuyến và Netfilter của host. Chế độ BPF host routing yêu cầu nhân Linux $\geq$ 5.10.



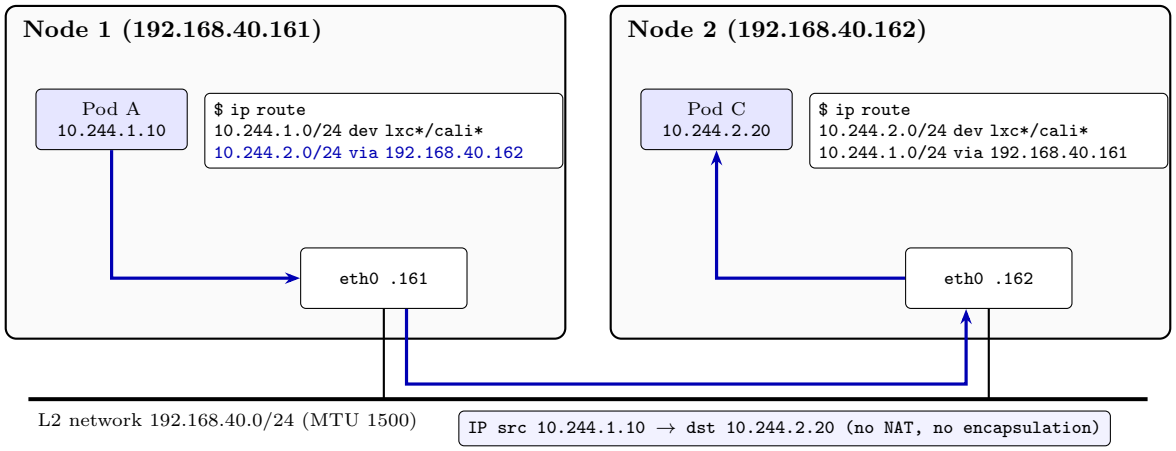
Hình 4: Đường đi gói tin giữa hai Pod trên cùng một node: Netfilter (trên) và eBPF host-routing (dưới).

3.4.2 Inter-node: định tuyến trực tiếp

Giữa các node, CNI có thể dựng một **mạng overlay** (VXLAN, Geneve, IP-in-IP) bọc gói tin của Pod trong một gói UDP/IP mang địa chỉ node. Cách này chạy được trên mọi hạ tầng nhưng thêm khoảng 50 byte header cho mỗi gói VXLAN (MTU trong Pod giảm còn khoảng 1450) và tốn CPU cho đóng gói/bóc gói. Khi các node cùng một mạng có thể học được dải Pod CIDR, có thể bỏ overlay và **định tuyến trực tiếp** [2]:

- **Calico** chạy daemon BGP (BIRD) trên mỗi node để quảng bá dải Pod CIDR của node cho các node khác.
- **Cilium** ở chế độ native routing, với các node cùng mạng L2, chỉ cần cài route trực tiếp tới dải Pod của từng node (`autoDirectNodeRoutes`).

Không có header đóng gói nên MTU gốc 1500 được giữ nguyên (Hình 5). Đồ án dùng định tuyến trực tiếp cho cả hai cụm để khác biệt đo được phản ánh cách xử lý Service và chính sách, không phải chi phí overlay.



Hình 5: Định tuyến trực tiếp giữa hai node: mỗi node có route tới dải Pod CIDR của node kia qua IP của node đó (Calico học qua BGP, Cilium cài bằng `autoDirectNodeRoutes`); gói tin giữ nguyên IP Pod nguồn/đích, không NAT và không đóng gói.

3.5 Cân bằng tải Service: kube-proxy iptables và Cilium eBPF

Mục này là cơ sở lý thuyết cho thí nghiệm **E1**.

kube-proxy ở chế độ iptables. kube-proxy biểu diễn mỗi Service thành một cây chuỗi trong bảng `nat`:

- `KUBE-SERVICES`: chuỗi gốc được gọi từ `PREROUTING` và `OUTPUT`, chứa **một luật cho mỗi cặp (ClusterIP, cổng)**. Gói tin được so khớp tuần tự với từng luật.
- `KUBE-SVC-<hash>`: mỗi Service một chuỗi, chọn ngẫu nhiên một backend bằng module `statistic` (xác suất $1/n$, $1/(n-1)$, ...).
- `KUBE-SEP-<hash>`: mỗi endpoint một chuỗi, thực hiện `DNAT` sang IP:cổng của Pod backend.

Listing 1: Ví dụ cấu trúc chuỗi luật của một Service trong bảng nat (rút gọn).

```
-A KUBE-SERVICES -d 10.233.12.34/32 -p tcp --dport 80 -j KUBE-SVC-ABCD
-A KUBE-SVC-ABCD -m statistic --mode random --probability 0.5 -j KUBE-SEP-1111
-A KUBE-SVC-ABCD -j KUBE-SEP-2222
-A KUBE-SEP-1111 -p tcp -j DNAT --to-destination 10.244.1.20:8080
-A KUBE-SEP-2222 -p tcp -j DNAT --to-destination 10.244.2.21:8080
```

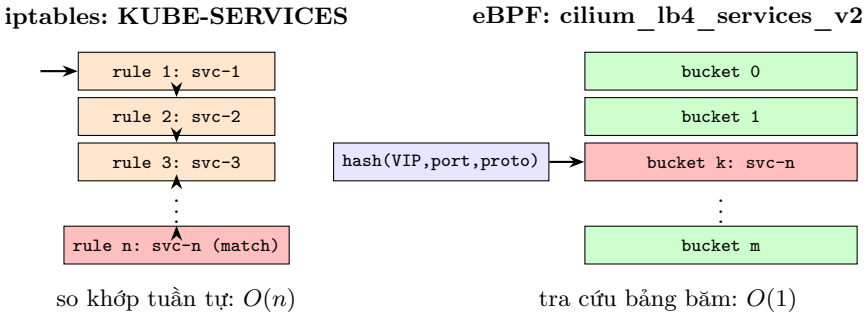
Hệ quả là:

- **Độ phức tạp tra cứu** $O(n)$ theo số Service: trong trường hợp xấu, gói tin SYN của một kết nối mới phải đi qua hàng nghìn luật trong `KUBE-SERVICES` trước khi gặp luật khớp. Chi phí này chỉ phát sinh khi thiết lập kết nối (các gói sau dùng conntrack), nên lộ rõ với tải kiểu request/response trên kết nối mới (TCP_CRR).
- **Chi phí cập nhật:** kube-proxy đồng bộ bằng cách sinh lại luật và nạp qua `iptables-restore`. Thao tác này giữ khoá toàn cục của xtables, và thời gian của nó tăng theo kích thước bảng. Từ Kubernetes 1.27, kube-proxy mặc định chỉ ghi lại các chuỗi thay đổi (partial sync), nhưng bảng luật vẫn được xử lý như một khối lớn.
- **Conntrack:** mọi kết nối qua Service đều tạo mục trong bảng conntrack dùng chung (`nf_conntrack`). Khi tốc độ tạo kết nối rất cao, tranh chấp khoá và giới hạn `nf_conntrack_max` có thể gây tăng tải CPU hoặc mất gói.

Cilium kube-proxy replacement (KPR). Khi bật `kubeProxyReplacement`, kube-proxy bị gỡ bỏ và Cilium hiện thực Service trực tiếp bằng eBPF [3]:

- Service và backend được lưu trong các **BPF hash map** (`cilium_lb4_services_v2`, `cilium_lb4_backends_v3`). Việc tra cứu theo khoá (ClusterIP, cổng, giao thức) có độ phức tạp trung bình $O(1)$, không phụ thuộc số Service trong cụm.

- **Socket-level load balancing:** với lưu lượng xuất phát từ Pod trong cụm, chương trình eBPF tại cgroup hook `connect()` dịch ClusterIP sang IP backend ngay ở tầng socket. Gói tin được tạo ra đã mang địa chỉ backend, nên không cần NAT theo từng gói. Lưu lượng từ bên ngoài (NodePort, LoadBalancer) được xử lý tại TC.
- **Cập nhật gia tăng:** khi Service/Endpoint thay đổi, cilium-agent chỉ cập nhật đúng các phần tử liên quan trong map bằng thao tác nguyên tử, không nạp lại toàn bộ bảng.
- Cilium dùng bảng conntrack riêng dạng BPF LRU hash map (`cilium_ct4_global`) thay vì `nf_conntrack`.



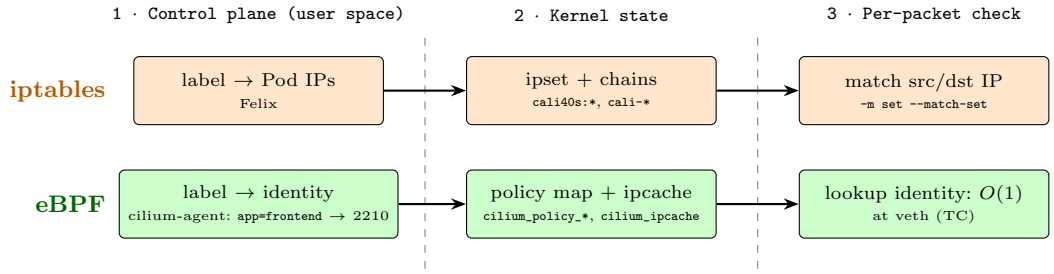
Hình 6: Tra cứu Service: duyệt tuần tự chuỗi iptables và tra cứu một lần trong BPF hash map.

3.6 Thực thi NetworkPolicy: iptables và eBPF

Mục này là cơ sở lý thuyết cho thí nghiệm **E2**. Đối tượng so sánh là hai **cơ chế datapath**, iptables và eBPF; Calico và Cilium chỉ là hai hiện thực cụ thể mà nhóm chọn để đo. NetworkPolicy của Kubernetes mô tả quy tắc cho phép lưu lượng vào/ra dựa trên label selector (Pod, namespace) và cổng. Vì gói tin chỉ mang địa chỉ IP chứ không mang label, mọi hiện thực đều phải chuyển đổi *label → thứ có thể kiểm tra trên gói tin*, theo cùng ba bước:

1. **Mặt phẳng điều khiển (user space):** một tác tử trên node theo dõi Pod và NetworkPolicy, rồi dịch label thành dữ liệu mà datapath dùng được.
2. **Trạng thái trong nhân:** dữ liệu đó được nạp vào cấu trúc của datapath (ipset và chuỗi luật với iptables; BPF map với eBPF).
3. **Kiểm tra từng gói:** datapath so khớp gói tin với trạng thái ở bước 2.

Khác biệt nằm ở *thứ được dịch ra ở bước 1*: với iptables là danh sách địa chỉ IP, với eBPF là một định danh bảo mật (security identity) độc lập với IP (Hình 7).



Hình 7: Ba bước thực thi NetworkPolicy: iptables dịch label thành danh sách IP của Pod (ipset và chuỗi `cali-*`), còn eBPF dịch label thành security identity và tra cứu trong policy map.

iptables (hiện thực: Calico): khớp theo địa chỉ IP. Trên mỗi node, tác tử **Felix** của Calico (user space) theo dõi Pod, NetworkPolicy và chuyển mỗi selector thành một **ipset** (kiểu `hash:ip`/`hash:net`, tên dạng `cali40s:...`) chứa IP của các Pod đang khớp label. Chính sách trở thành các chuỗi iptables (`cali-fw-<iface>`, `cali-tw-<iface>`, `cali-pi-*`, `cali-po-*`) với luật dạng `-m set --match-set cali40s:... src -j ACCEPT`. Việc tra cứu trong ipset là băm, nhưng số chuỗi/luật vẫn tăng theo số chính sách. Quan trọng hơn, **chính sách**

bị ràng buộc với IP tạm thời: mỗi khi Pod được tạo, huỷ hoặc đổi IP, Felix trên *mọi* node phải cập nhật ipset tương ứng. Trong khoảng trễ đồng bộ, Pod mới có thể bị chặn nhầm hoặc IP cũ được tái sử dụng có thể bị cho phép nhầm [6].

eBPF (hiện thực: Cilium): khớp theo định danh bảo mật. Datapath eBPF tách chính sách khỏi địa chỉ IP [5, 6]:

- **Định danh đến từ đâu:** security identity *không* do eBPF sinh ra. Tác tử **cilium-agent** (user space) cấp cho mỗi tập label liên quan đến bảo mật của Pod một identity dạng số, ví dụ `app=frontend` → 2210, và lưu nó thành tài nguyên `CiliumIdentity` để mọi node dùng chung. Mọi Pod có cùng tập label dùng chung một định danh, bất kể số bản sao. Một số định danh được dành riêng (ví dụ `host`, `world`).
- **Vai trò của eBPF:** lưu identity trong BPF map và kiểm tra nó trên từng gói. Với mỗi endpoint, agent tính trước một **policy map** có khoá (định danh nguồn/đích, cổng, giao thức, chiều) và giá trị cho phép/từ chối. Chương trình eBPF tại veth tra cứu map này với độ phức tạp $O(1)$.
- Định danh của gói tin được xác định qua `cilium_ipcache` (IP → định danh) hoặc lấy trực tiếp từ header VNI khi chạy chế độ tunnel.
- Khi Pod mới có tập label đã tồn tại được tạo ra, định danh đã có sẵn nên **policy map không cần thay đổi**; chỉ có ánh xạ IP → định danh trong ipcache cần được phân phối. Nhờ vậy chi phí và độ trễ đồng bộ không phụ thuộc vào số chính sách.
- **Chính sách L7:** với `CiliumNetworkPolicy`, lưu lượng khớp quy tắc L7 được chuyển hướng tới proxy Envoy trên node để kiểm tra HTTP method/path, gRPC, Kafka, DNS, vượt ra ngoài phạm vi L3/L4 của NetworkPolicy chuẩn.

Bảng 1 tóm tắt sự khác biệt.

	iptables (Calico)	**eBPF (Cilium)**
Đơn vị nhận diện	Địa chỉ IP của Pod	Security identity (số) từ label
Ai tạo ra (user space)	Felix: selector → danh sách IP	cilium-agent: tập label → identity
Cấu trúc trong nhân	ipset + chuỗi `cali-*`	`cilium_policy_*`, `cilium_ipcache`
Khi Pod mới xuất hiện	Cập nhật ipset trên mọi node	Chỉ cập nhật ipcache; policy map giữ nguyên
Phạm vi	L3/L4	L3/L4 và L7 (qua Envoy)
Công cụ kiểm tra	`ipset list`, `iptables-save`	`cilium identity list`, `cilium bpf policy get`

Bảng 1: So sánh cơ chế thực thi NetworkPolicy giữa hai datapath (trong ngoặc: hiện thực được dùng trong đồ án).

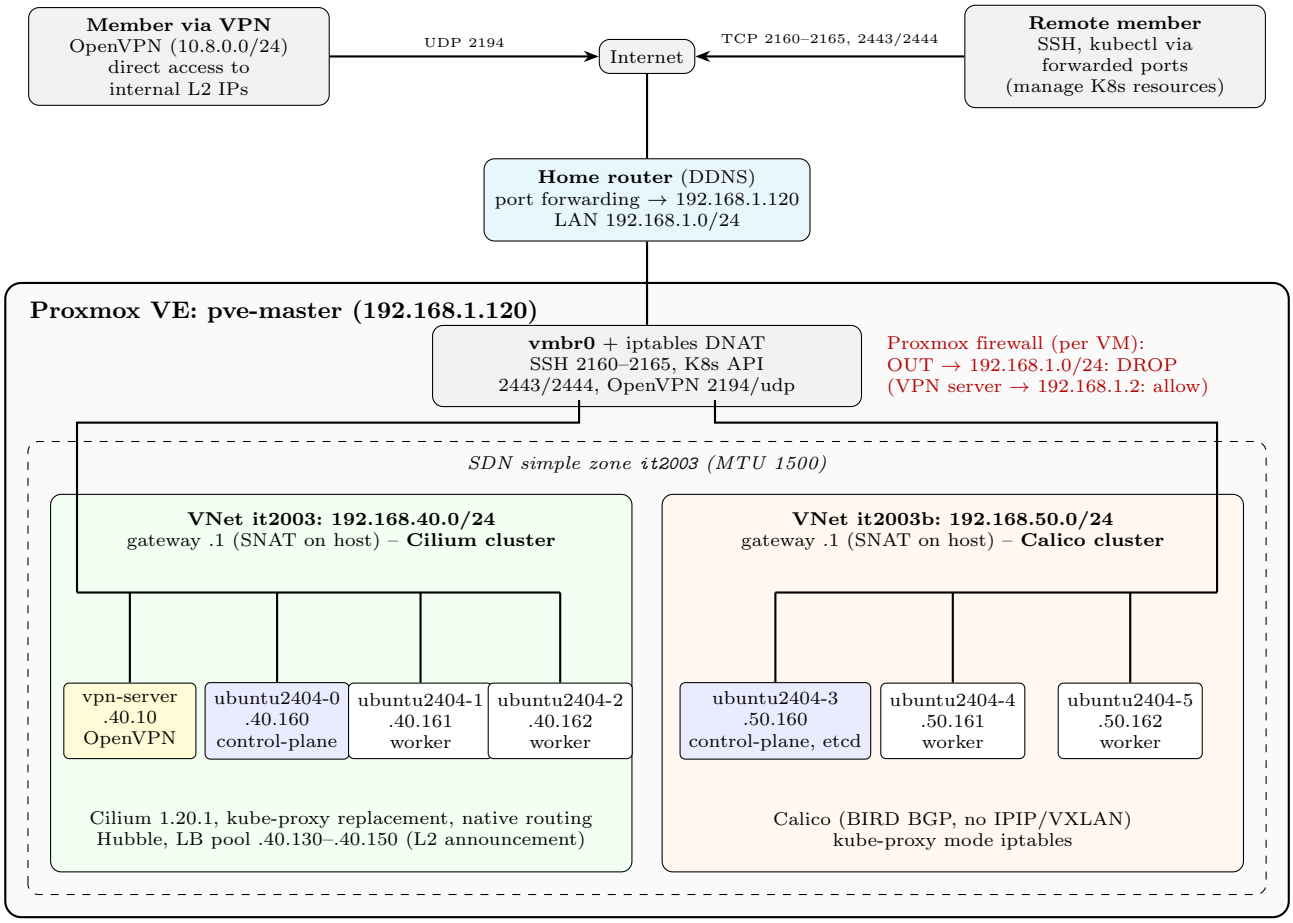
4 Phương pháp Triển Khai

4.1 Hạ tầng và topology mạng

Môi trường thực nghiệm là **homelab** của một thành viên: một máy chủ Proxmox VE (`pve-master`) nằm sau router gia đình, truy cập từ xa qua DDNS và chuyển tiếp cổng, nên cả nhóm dùng chung mà không cần thuê cloud. Hạ tầng được mô tả dưới dạng mã (Infrastructure as Code):

- **Terraform** (provider `bpg/proxmox`) tạo vùng mạng SDN, các VNet/subnet, máy ảo từ cloud image Ubuntu 24.04 và luật tường lửa Proxmox cho từng máy ảo.
- **Ansible** chuẩn bị hệ điều hành (tắt swap, tạo người dùng) và cấu hình DNAT trên host Proxmox để truy cập từ bên ngoài.
- **Kubespray** cài đặt Kubernetes; với cụm Cilium, CNI được cài bằng **Helm** (chart Cilium 1.20.1).

Thay vì cài đặt lại cùng một cụm giữa hai cấu hình, nhóm triển khai **hai cụm Kubernetes song song**, mỗi cụm ba node, trên hai subnet tách biệt. Các máy ảo của hai cụm có cấu hình phần cứng giống hệt nhau (4 vCPU kiểu `host`, 4 GiB RAM, đĩa virtio 120 GB) và cùng chạy trên một host vật lý. Nhờ vậy hai datapath được đo trong cùng điều kiện, và có thể chạy thí nghiệm hoặc demo trên cả hai cụm cùng lúc. Hình 8 mô tả topology tổng thể.



Hình 8: Topology hạ tầng homelab: thành viên truy cập qua Internet → router gia đình (DDNS, port forwarding) → host Proxmox, nơi hai cụm Kubernetes chạy trên hai VNet riêng biệt.

Danh sách máy ảo. Bảng 2 liệt kê các máy ảo. Mọi máy ảo dùng cloud image Ubuntu 24.04 (nhân 6.8.0-142-generic) và được cấp IP tĩnh qua cloud-init.

Máy ảo	IP	Cụm	Vai trò	vCPU / RAM / Disk
`it-2003-ubuntu2404-0`	192.168.40.160	Cilium	control plane (`k8sServiceHost`)	4 / 4 GiB / 120 GB
`it-2003-ubuntu2404-1`	192.168.40.161	Cilium	worker	4 / 4 GiB / 120 GB
`it-2003-ubuntu2404-2`	192.168.40.162	Cilium	worker	4 / 4 GiB / 120 GB
`it-2003-ubuntu2404-3`	192.168.50.160	Calico	control plane, etcd	4 / 4 GiB / 120 GB
`it-2003-ubuntu2404-4`	192.168.50.161	Calico	worker	4 / 4 GiB / 120 GB
`it-2003-ubuntu2404-5`	192.168.50.162	Calico	worker	4 / 4 GiB / 120 GB
`it-2003-vpn-server`	192.168.40.10	–	OpenVPN gateway (client 10.8.0.0/24)	2 / 512 MiB / 30 GB

Bảng 2: Danh sách máy ảo của môi trường thực nghiệm.

Cấu hình mạng của hai cụm. Hai cụm chỉ khác nhau ở lớp mạng Kubernetes (Bảng 3). Cả hai đều dùng **định tuyến trực tiếp, không overlay** (Mục 3.4). Calico quảng bá route của Pod qua BGP (daemon BIRD) với IPIP và VXLAN tắt. Cilium chạy ở `routingMode: native`, và `autoDirectNodeRoutes` cài route trực tiếp

giữa các node trong cùng mạng L2. Vì không có header đóng gói, MTU 1500 của VNet được giữ nguyên, và sự khác biệt đo được phản ánh chủ yếu cách xử lý Service và chính sách (Mục 3.5) chứ không phải chi phí overlay.

Thành phần	Cụm Calico (192.168.50.0/24)	Cụm Cilium (192.168.40.0/24)
CNI	Calico (Kubespray), iptables dataplane	Cilium 1.20.1 (Helm)
Service datapath	kube-proxy, `kube_proxy_mode: iptables`	`kubeProxyReplacement: true` (không có kube-proxy)
Định tuyến Pod	BGP (BIRD), IPIP/VXLAN: Never	Native routing, `autoDirectNodeRoutes`
Masquerade	iptables	eBPF (`bpf.masquerade`)
Pod CIDR	10.244.0.0/16, khối /26 (Calico IPAM)	10.244.0.0/16, /24 mỗi node (IPAM `kubernetes`)
Service CIDR	110.96.0.0/12	10.96.0.0/12
LoadBalancer	Không (chỉ ClusterIP/NodePort)	LB-IPAM + L2 announcement, pool 192.168.40.130–150
Giám sát / quan sát	kube-prometheus-stack, Loki, Alloy, metrics-server	kube-prometheus-stack, Loki, Alloy, metrics-server; Hubble, Hubble Relay, Hubble UI (192.168.40.132); Tetragon
Kubernetes	v1.35.8 (mọi node)	v1.35.7 (control plane), v1.35.8 (worker)
containerd	2.3.5	2.2.1
Hệ điều hành / nhân	Ubuntu 24.04.5, 6.8.0-142-generic	Ubuntu 24.04.4, 6.8.0-142-generic

Bảng 3: So sánh cấu hình hai cụm Kubernetes.

Truy cập và cô lập. Router gia đình chuyển tiếp cổng tới host Proxmox, và các luật DNAT trên `vmbr0` chuyển tiếp tiếp tới máy ảo (Hình 8). Thành viên quản trị cụm bằng SSH và `kubectl` qua các cổng này (Kubernetes API: 2443 cho Cilium, 2444 cho Calico). Khi chạy thí nghiệm và demo, thành viên kết nối OpenVPN để truy cập trực tiếp các IP nội bộ của cụm, kể cả IP LoadBalancer do Cilium quảng bá. Mỗi VNet ra Internet qua SNAT trên host, và tường lửa Proxmox của từng máy ảo chặn mọi lưu lượng tới mạng LAN gia đình 192.168.1.0/24.

Thiết lập và quy trình của từng thí nghiệm được trình bày ở đầu mục kết quả tương ứng (E1: Mục 5.1).

5 Kết Quả Thực Nghiệm và Đánh Giá

5.1 E1 – Hiệu năng Service theo quy mô

Câu hỏi và giả thuyết. Thí nghiệm E1 trả lời RQ1: *độ trễ thiết lập kết nối tới một Service thay đổi thế nào khi tổng số Service trong cụm tăng?* Theo Mục 3.5, kube-proxy ở chế độ iptables so khớp tuần tự chuỗi `KUBE-SERVICES` cho gói tin đầu tiên của mỗi kết nối ($O(n)$), nên độ trễ dự kiến tăng theo số Service. Cilium tra cứu Service trong BPF hash map ($O(1)$), nên độ trễ dự kiến không đổi.

Thiết lập đo. Thí nghiệm chạy trên hai cụm mô tả ở Mục 4.1, với cùng một bộ manifest và script cho cả hai cụm:

- **Service giả:** tạo $N \in \{100;\ 2\,080;\ 4\,060;\ 6\,040;\ 8\,020;\ 10\,000\}$ Service giả. Mỗi Service chọn cùng 3 Pod `pause` làm endpoint, nên kube-proxy sinh đầy đủ cây chuỗi (`KUBE-SERVICES` → `KUBE-SVC` → 3 × `KUBE-SEP`) gồm đúng 11 luật nat cho mỗi Service (Listing 2): 1 luật trong `KUBE-SERVICES` so khớp ClusterIP:80; 4 luật trong `KUBE-SVC` (1 luật `KUBE-MARK-MASQ` cho lưu lượng đến từ ngoài dải Pod `10.244.0.0/16`, cộng 3 luật nhảy tới 3 `KUBE-SEP` với xác suất 1/3, 1/2 và phần còn lại); và 3 × 2 luật trong các `KUBE-SEP` (1 luật `KUBE-MARK-MASQ` cho trường hợp hairpin, tức Pod gọi Service trỏ về chính nó, cộng 1 luật `DNAT`).

Listing 2: Toàn bộ 11 luật nat của Service `e1-svc-00001` trích từ `iptables-save -t nat` trên node client (đã bỏ `-m comment`).

```
-A KUBE-SERVICES -d 110.97.109.12/32 -p tcp -m tcp --dport 80 -j KUBE-SVC-OHDNYIY65FBVL4TT
-A KUBE-SVC-OHDNYIY65FBVL4TT ! -s 10.244.0.0/16 -d 110.97.109.12/32 -p tcp -m tcp --dport 80 -j KUBE-MARK-
    MASQ
-A KUBE-SVC-OHDNYIY65FBVL4TT -m statistic --mode random --probability 0.33333333349 -j KUBE-SEP-
    DHKL3RFXPGHTGTPE
-A KUBE-SVC-OHDNYIY65FBVL4TT -m statistic --mode random --probability 0.50000000000 -j KUBE-SEP-
    XPQ5ZHQALF6TTRRL
-A KUBE-SVC-OHDNYIY65FBVL4TT -j KUBE-SEP-LJC42FAZ5AM773HU
```

```
-A KUBE-SEP-DHKL3RFXPGHTGTPE -s 10.244.155.202/32 -j KUBE-MARK-MASQ
-A KUBE-SEP-DHKL3RFXPGHTGTPE -p tcp -m tcp -j DNAT --to-destination 10.244.155.202:8080
-A KUBE-SEP-XPQ5ZHQALF6TTRRL -s 10.244.155.203/32 -j KUBE-MARK-MASQ
-A KUBE-SEP-XPQ5ZHQALF6TTRRL -p tcp -m tcp -j DNAT --to-destination 10.244.155.203:8080
-A KUBE-SEP-LJC42FAZ5AM773HU -s 10.244.60.105/32 -j KUBE-MARK-MASQ
-A KUBE-SEP-LJC42FAZ5AM773HU -p tcp -m tcp -j DNAT --to-destination 10.244.60.105:8080
```

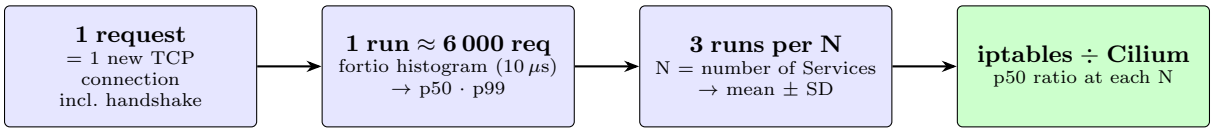
Trên Cilium, mỗi Service giả chiếm đúng 5 dòng trong LB map (Listing 3), mỗi dòng là một key `(ClusterIP, cổng, protocol, slot)`: 1 dòng slot 0 của `ClusterIP:80/TCP` là dòng frontend gốc, không trỏ tới backend nào (`0.0.0.0:0`) mà lưu số backend và RevNAT ID dùng để đổi địa chỉ nguồn của gói tin trả lời về lại ClusterIP; 3 dòng slot 1–3, mỗi dòng trỏ tới một Pod backend; và 1 dòng wildcard `ClusterIP:0/ANY` mà Cilium tự thêm cho mỗi ClusterIP, để gói tin tới ClusterIP trên cổng hoặc protocol không khai báo bị drop. Với một kết nối mới, datapath tra slot 0 để biết số backend, chọn ngẫu nhiên một slot trong 1–3 rồi tra tiếp slot đó, tức hai lần tra hash map, không phụ thuộc vào số Service.

Listing 3: Toàn bộ 5 dòng LB map của một Service giả, trích từ `cilium-dbg bpf lb list` trên node client (sắp xếp lại theo slot). Cột trái: frontend (slot); cột phải: backend (RevNAT ID) (slot).

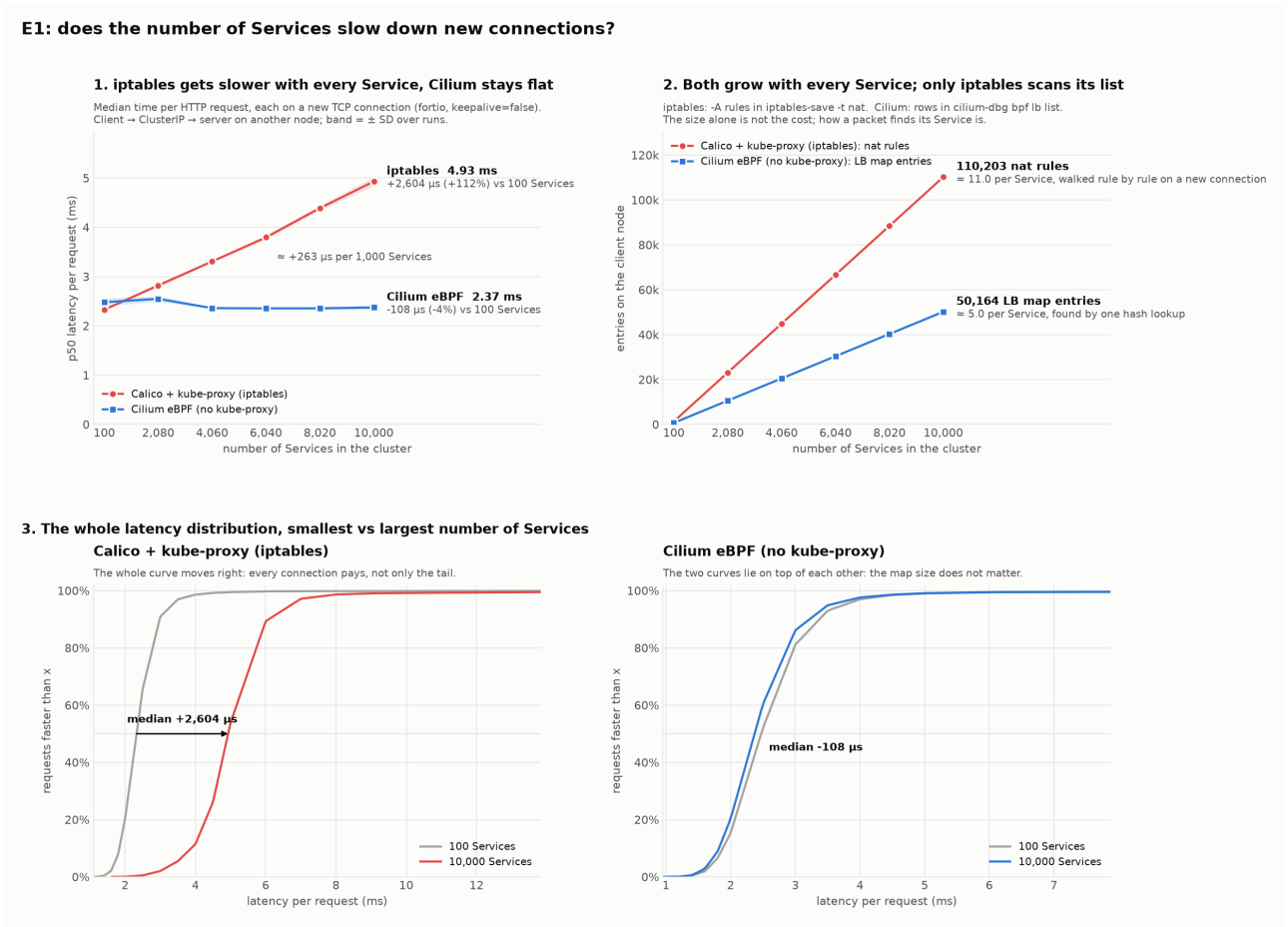
```
SERVICE ADDRESS              BACKEND ADDRESS (REVNAT_ID) (SLOT)
10.100.108.158:80/TCP (0)    0.0.0.0:0 (15008) (0) [ClusterIP, non-routable]
10.100.108.158:80/TCP (1)    10.244.0.120:8080/TCP (15008) (1)
10.100.108.158:80/TCP (2)    10.244.0.233:8080/TCP (15008) (2)
10.100.108.158:80/TCP (3)    10.244.2.51:8080/TCP (15008) (3)
10.100.108.158:0/ANY (0)     0.0.0.0:0 (0) (0) [ClusterIP, non-routable]
```

- **Đo:** fortio 1.75.3 (client và server nằm trên hai worker khác nhau: Cilium node-2 → node-1, Calico node-5 → node-4) gọi trực tiếp ClusterIP của fortio-server với `-keepalive=false`, tức mỗi yêu cầu mở một kết nối TCP mới. Mỗi lần chạy: 200 yêu cầu/giây, 4 kết nối song song, 30 giây, có 5 giây làm nóng. Độ trễ đo được gồm cả bắt tay TCP và thời gian phản hồi của ứng dụng.
- **Đồng bộ:** sau khi tạo Service, script chờ đủ EndpointSlice, chờ số luật/entry trên node client ổn định, rồi chờ thêm 20 giây trước khi đo. Mỗi giá trị N được đo 3 lần.
- **Kích thước datapath:** với iptables là số luật trong `iptables-save -t nat`; với Cilium là số dòng trong `cilium-dbg bpf lb list`, đọc trên node client.
- **Tải nền:** hai cụm chạy cùng một bộ ứng dụng nền (kube-prometheus-stack, Loki, Alloy, metrics-server, microservices demo, DVWA và ứng dụng của E2). Tetragon (dùng cho E3) được tạm dừng trên cụm Cilium trong lúc đo, vì riêng nó chiếm khoảng 1.9 lõi CPU trên node server. Khi rảnh, mỗi node của cả hai cụm dùng khoảng 0.5 lõi. Toàn bộ quy trình được điều khiển bằng script `e1/scripts/run-e1.py`.

Tổng cộng có 36 lần chạy (6 giá trị N × 3 lần × 2 cụm). Hình 9 tóm tắt cách một con số trong kết quả E1 được tạo ra.



Hình 9: Quy trình tạo số liệu E1: mỗi yêu cầu mở một kết nối TCP mới; mỗi lần chạy cho p50/p99 từ histogram của fortio; mỗi giá trị N lấy trung bình ± độ lệch chuẩn của 3 lần chạy; cuối cùng so sánh tỉ lệ p50 giữa hai datapath.



Hình 10: Kết quả E1. (1) Độ trễ trung vị (p50) mỗi yêu cầu (kết nối mới) theo số Service; mỗi điểm là trung bình 3 lần chạy, dải màu là ± độ lệch chuẩn. (2) Kích thước bảng của datapath trên node client theo số Service: số luật nat (iptables) và số dòng LB map (Cilium). (3) Phân phối tích luỹ (CDF) độ trễ ở số Service nhỏ nhất và lớn nhất của từng datapath, dựng từ histogram của fortio.

	Kích thước bảng		p50 (ms)		p99 (ms)		
N Service	iptables	Cilium	iptables	Cilium	iptables	Cilium	p50 ipt/cil
100	1 303	664	2.33 ± 0.01	2.48 ± 0.08	4.28	6.59	0.94
2 080	23 083	10 564	2.82 ± 0.04	2.55 ± 0.06	4.95	5.24	1.11
4 060	44 863	20 464	3.31 ± 0.01	2.36 ± 0.02	6.08	5.03	1.40
6 040	66 643	30 364	3.80 ± 0.04	2.35 ± 0.01	6.51	5.80	1.61
8 020	88 423	40 264	4.39 ± 0.05	2.36 ± 0.03	6.82	4.48	1.86
10 000	110 203	50 164	4.93 ± 0.09	2.37 ± 0.04	9.93	4.96	2.08

Bảng 4: Tổng hợp kết quả E1 (trung bình ± độ lệch chuẩn của 3 lần chạy). Kích thước bảng: số luật nat (iptables) và số dòng LB map (Cilium) trên node client.

Nhận xét.

1. **Cilium không phụ thuộc số Service.** Độ trễ trung vị (p50) là 2.48 ms ở 100 Service và 2.37 ms ở 10 000 Service. Trên toàn dải đo, p50 nằm trong khoảng 2.35–2.55 ms; độ dốc hồi quy khoảng −0.016 ms cho mỗi 1 000 Service, nằm trong mức dao động giữa các lần chạy.

2. **iptables tăng tuyến tính.** p50 tăng từ 2.33 ms lên 4.93 ms (+112%). Kết quả khớp gần như hoàn toàn với một đường thẳng ($r = 0.999$), tương ứng khoảng +0.26 ms mỗi 1 000 Service hay +239 μs mỗi 10 000 luật nat. Hình dạng tuyến tính này chính là chi phí duyệt chuỗi $O(n)$ đã phân tích ở Mục 3.5. Ở 10 000 Service, iptables chậm hơn Cilium 2.08 lần, tức chậm thêm 2.56 ms cho mỗi kết nối mới.

3. **Cách tra cứu quyết định, không phải kích thước bảng.** Bảng của cả hai datapath đều tăng tuyến tính theo số Service (Hình 10, biểu đồ 2), nhưng chỉ iptables trả giá cho điều đó. Ở kích thước bảng tương đương, Cilium với 50 164 entry cho p50 2.37 ms, còn iptables với 44 863 luật cho 3.31 ms (Hình 10, biểu đồ 1 và 2).

4. **Toàn bộ phân phối dịch chuyển.** Với iptables, cả đường CDF ở 10 000 Service dịch sang phải so với 100 Service (trung vị +2.6 ms), tức mọi kết nối đều chịu thêm chi phí chứ không riêng phần đuôi. Với Cilium, hai đường CDF gần như trùng nhau (Hình 10, biểu đồ 3).

5. **Chi phí nằm ở bước thiết lập kết nối.** Log của fortio ghi riêng thời gian `connect()` (bắt tay TCP) của từng kết nối. Với iptables, thời gian này tăng từ trung bình 1.10 ms ở 100 Service lên 3.30 ms ở 10 000 Service (+2.2 ms), chiếm phần lớn mức tăng +2.63 ms của độ trễ trung bình mỗi yêu cầu. Với Cilium, thời gian `connect()` giữ ở 1.08 ms và 1.06 ms. Điều này khớp với Mục 3.5: chỉ gói SYN đầu tiên phải duyệt `KUBE-SERVICES`, các gói sau đi theo conntrack.

6. **Độ trễ đuôi (p99).** Ở quy mô lớn, xu hướng giống p50: tại 10 000 Service, p99 của iptables là 9.93 ms so với 4.96 ms của Cilium. Giá trị p99 này của iptables dao động mạnh giữa 3 lần chạy (7.65, 10.19 và 11.94 ms), nên chỉ nên đọc như xu hướng, không phải con số chính xác. Dưới khoảng 3 000 Service, iptables lại có đuôi ngắn hơn, và p99 của Cilium dao động mạnh hơn giữa các lần chạy. Lợi thế của eBPF vì vậy nằm ở khả năng mở rộng, không phải mọi yêu cầu đều nhanh hơn ở mọi quy mô.

7. **Vị trí luật khớp.** Vị trí luật của fortio-server trong `KUBE-SERVICES` tăng theo số Service (từ 46 lên 8 705), cùng chiều với độ trễ. Các yếu tố khác như việc node server cũng duyệt `KUBE-SERVICES` cho kết nối đã DNAT, hay áp lực bộ nhớ đệm CPU, chưa được tách riêng; kết luận chính không đổi: chi phí của iptables tăng theo tổng số Service.

Đánh giá. Kết quả **ủng hộ giả thuyết**: trong môi trường thực nghiệm này, chi phí định tuyến Service của Cilium eBPF không đổi khi cụm tăng từ 100 lên 10 000 Service. Với iptables, chi phí tăng tuyến tính và độ trễ trung vị tăng gấp đôi ở 10 000 Service. Toàn bộ 216 000 yêu cầu đều thành công ở mức tải 200 yêu cầu/giây, nên khác biệt giữa hai datapath chỉ nằm ở độ trễ. Điều này xác nhận phân tích tra cứu Service $O(n)$ và $O(1)$ ở Mục 3.5.

Giới hạn của E1.

- Mỗi điểm chỉ có 3 lần chạy ở một mức tải (200 yêu cầu/giây): đủ để thấy xu hướng mạnh như trên, nhưng chưa đo được hiệu ứng nhỏ, thông lượng tối đa hay mức sử dụng CPU.
- Độ trễ tuyệt đối gồm cả bắt tay TCP (trung bình khoảng 1.1 ms ở 100 Service trên cả hai cụm) và thời gian gửi yêu cầu, nhận phản hồi, vì vậy chênh lệch tuyệt đối (2.56 ms) có ý nghĩa hơn tỉ lệ.
- Tetragon được tạm dừng trong lúc đo, nên chi phí của nó lên đường kết nối không có trong kết quả E1.
- Chưa đo thời gian cập nhật luật khi thêm hoặc xoá Service, một điểm yếu khác của iptables ở quy mô lớn.
- Với Cilium, thí nghiệm đo đường socket-level load balancing (dịch địa chỉ tại `connect()`), là cấu hình mặc định khi bật kube-proxy replacement. Đường xử lý theo từng gói tại TC chưa được đo riêng.
- Hai datapath chạy trên hai cụm riêng. Các máy ảo giống hệt nhau và cùng chạy trên một host, nhưng phiên bản containerd và bản vá hệ điều hành có khác biệt nhỏ (Bảng 3).

5.2 E2 – Biểu diễn và thực thi NetworkPolicy trên Cilium và Calico

Mục tiêu thí nghiệm. Trong E2, nhóm tìm hiểu cách Cilium và Calico lưu và áp dụng chính sách mạng, cũng như những thay đổi khi thêm Pod mới. Với Cilium, nhóm kiểm tra ba trường hợp: hạn chế kết nối trực tiếp giữa các dịch vụ của Online Boutique, chặn reverse shell từ DVWA và lọc yêu cầu HTTP theo đường dẫn API. Nhóm kiểm tra kết quả bằng trạng thái kết nối, log của `cilium-dbg monitor` và thông tin từ Hubble.

5.2.1 Cilium: cấu hình chính sách mạng

Ứng dụng Online Boutique nằm trong namespace `microservices-google-demo`; DVWA nằm trong namespace `dvwa`. Theo cấu hình cụm ở Mục 4.1, Cilium chạy native routing và thay thế kube-proxy. Các file cấu hình của Online Boutique gồm rule mặc định chặn kết nối, rule cho phép DNS và rule cho phép các dịch vụ gọi nhau theo chức năng của ứng dụng. Các tệp `CiliumNetworkPolicy` dùng `endpointSelector`, selector nguồn/đích và cổng; ví dụ `boutique-redis-cart` chọn Redis và khai báo ingress từ Cart tới TCP 6379. Cilium dùng các selector trong YAML để xác định endpoint và security identity cần áp dụng chính sách, như đã trình bày ở Mục 3.6.

Ở trạng thái ban đầu, namespace ứng dụng chưa có tài nguyên `CiliumNetworkPolicy`; lệnh `kubectl get cnp` trả về `No resources found`. Nhóm dùng trạng thái chưa có policy để so sánh với kết quả sau đó. Để kiểm tra policy có chặn kết nối hay không, nhóm xem kết quả truy cập và log của Cilium.

```
C:\Users\Phuong>kubectl get pod -n microservices-google-demo -o wide
NAME                                 READY   STATUS    RESTARTS      AGE   IP            NODE                 NOMINATED NODE   READINESS G
ATES
adservice-5f7c6d967f-l4fgw           1/1     Running   3 (32d ago)   63d   10.244.2.21   it-2003-ubuntu2404-2   <none>           <none>
cartservice-6959547fbb-8flnx         1/1     Running   3 (32d ago)   63d   10.244.1.41   it-2003-ubuntu2404-1   <none>           <none>
checkoutservice-77f79b594f-q77hz     1/1     Running   3 (32d ago)   63d   10.244.1.127  it-2003-ubuntu2404-1   <none>           <none>
currencyservice-84474685bc-fn4xs     1/1     Running   3 (32d ago)   63d   10.244.1.243  it-2003-ubuntu2404-1   <none>           <none>
frontend-7cdbb87b49-gjm7r            1/1     Running   0             7h2m  10.244.1.60   it-2003-ubuntu2404-1   <none>           <none>
paymentservice-6fb99579c7-hlxjm      1/1     Running   3 (32d ago)   63d   10.244.2.27   it-2003-ubuntu2404-2   <none>           <none>
productcatalogservice-85cbf65d89-dcp2s  1/1  Running   3 (32d ago)   63d   10.244.2.133  it-2003-ubuntu2404-2   <none>           <none>
redis-cart-6899b65948-g9m6g          1/1     Running   3 (32d ago)   63d   10.244.2.95   it-2003-ubuntu2404-2   <none>           <none>
shippingservice-669794b46f-q448k     1/1     Running   3 (32d ago)   63d   10.244.1.29   it-2003-ubuntu2404-1   <none>           <none>
thu-cartservice                      0/1     Running   0             8d    10.244.2.48   it-2003-ubuntu2404-2   <none>           <none>
thu-checkoutservice                  0/1     Running   0             8d    10.244.2.140  it-2003-ubuntu2404-2   <none>           <none>
thu-frontend                         0/1     Running   0             8d    10.244.1.85   it-2003-ubuntu2404-1   <none>           <none>
```

Hình 11: Danh sách Pod của Online Boutique và địa chỉ IP dùng trong phép thử: frontend `10.244.1.60`, Redis `10.244.2.95` và Payment `10.244.2.27`.

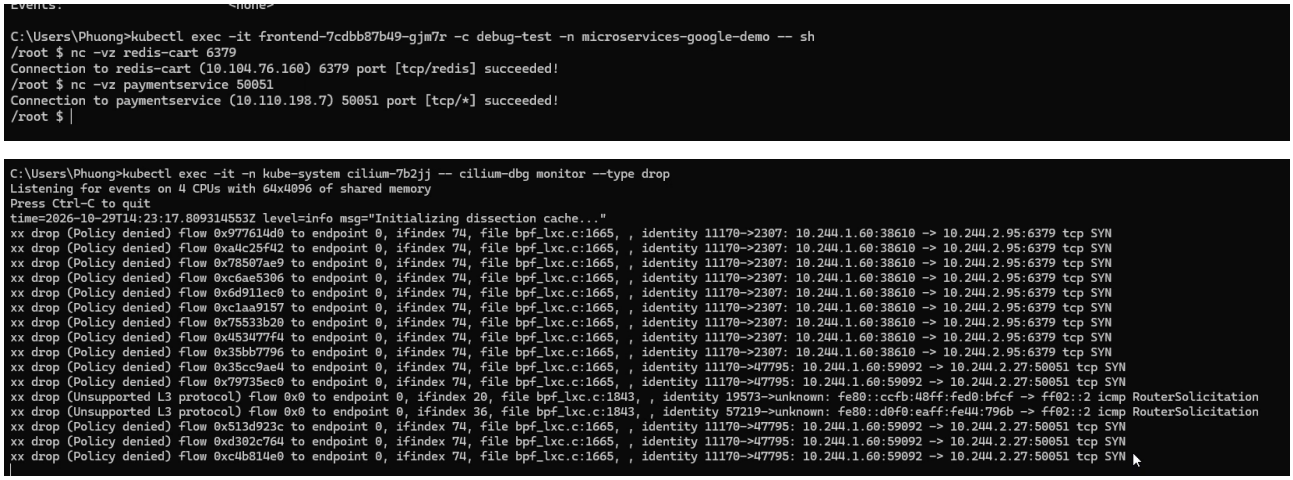
5.2.2 Cilium: giới hạn kết nối giữa các dịch vụ

Cách kiểm tra. Để mô phỏng trường hợp frontend bị chiếm quyền, nhóm sử dụng container hỗ trợ kiểm tra mạng `debug-test` trong Pod frontend để kết nối trực tiếp tới Redis và Payment. Ở lần kiểm tra ban đầu, hai lệnh `nc -vz redis-cart 6379` và `nc -vz paymentservice 50051` đều trả về `succeeded!`, cho thấy frontend kết nối TCP được tới cả hai dịch vụ.

Ở lần kiểm tra kết nối bị chặn, lệnh `cilium-dbg monitor --type drop` trên node chứa frontend ghi lại hai kết nối bị chặn:

- Redis: `10.244.1.60:38610` → `10.244.2.95:6379`, identity `11170` → `2307`, TCP SYN bị `Policy denied`.
- Payment: `10.244.1.60:59092` → `10.244.2.27:50051`, identity `11170` → `47795`, TCP SYN bị `Policy denied`.

Các gói SYN truyền lại cũng bị loại bỏ. Đối chiếu địa chỉ IP với danh sách Pod ở Hình 11 cho thấy nguồn là frontend, còn hai đích lần lượt là Redis và Payment. Trường `Policy denied` cho biết kết nối bị chặn do policy. Các sự kiện `Unsupported L3 protocol` của IPv6 Router Solicitation không liên quan đến hai kết nối TCP đang kiểm tra nên nhóm không xét các dòng này. Log cho biết gói tin bị chặn và identity của nguồn, đích, nhưng không ghi rõ tên policy gây chặn.

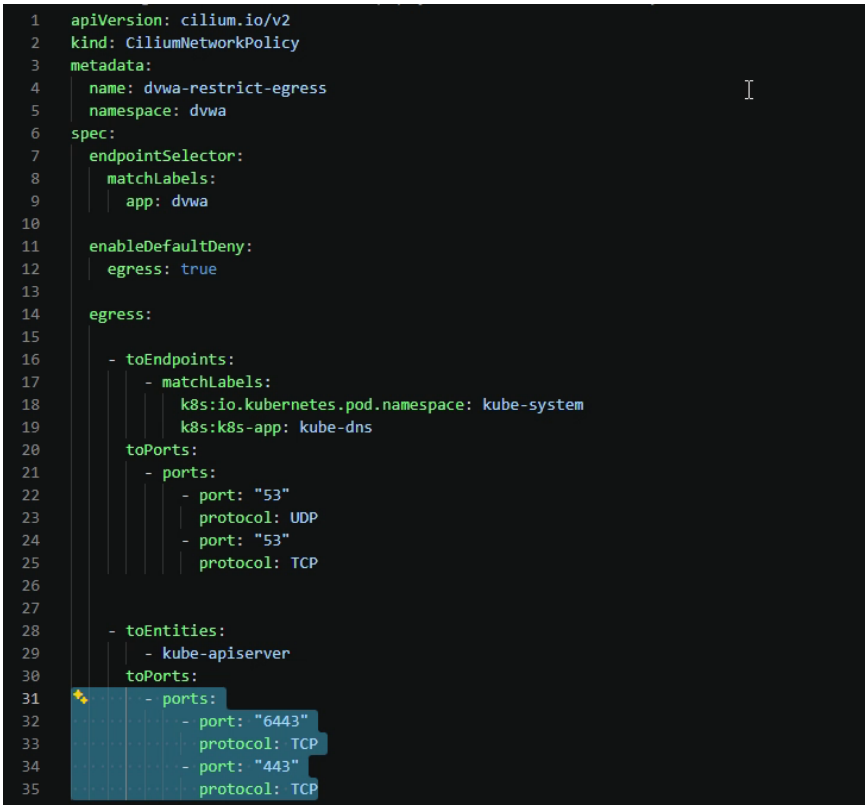


Hình 12: Kiểm tra kết nối từ frontend tới Redis và Payment: hai kết nối TCP thành công ở lần kiểm tra ban đầu (trên); các gói SYN bị từ chối với lý do `Policy denied` khi kiểm tra thực thi chính sách (dưới).

Theo cấu hình dự kiến, frontend truy cập Redis thông qua Cart và Payment thông qua Checkout. Kết quả kiểm thử cho thấy hai đường kết nối trực tiếp từ frontend tới Redis và Payment bị chặn. Nhóm chưa kiểm tra frontend gọi Cart và chưa chạy thử toàn bộ quy trình mua hàng, nên chưa kết luận được tất cả kết nối cần thiết của ứng dụng vẫn hoạt động.

5.2.3 Cilium: chặn kết nối reverse shell từ DVWA

Chính sách egress. Ở lần kiểm tra ban đầu, máy lắng nghe nhận được kết nối từ DVWA và mở shell dưới tài khoản `www-data`. Nhóm áp dụng policy `dvwa-restrict-egress` cho các Pod có nhãn `app=dvwa`. Policy bật `enableDefaultDeny.egress: true`, cho phép DNS tới `kube-dns` trong namespace `kube-system` trên UDP/TCP 53 và kết nối tới entity `kube-apiserver` trên TCP 443/6443. Địa chỉ `10.8.0.10:1234` dùng để nhận reverse shell không nằm trong danh sách được cho phép.



```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: dvwa-restrict-egress
  namespace: dvwa
spec:
  endpointSelector:
    matchLabels:
      app: dvwa

  enableDefaultDeny:
    egress: true

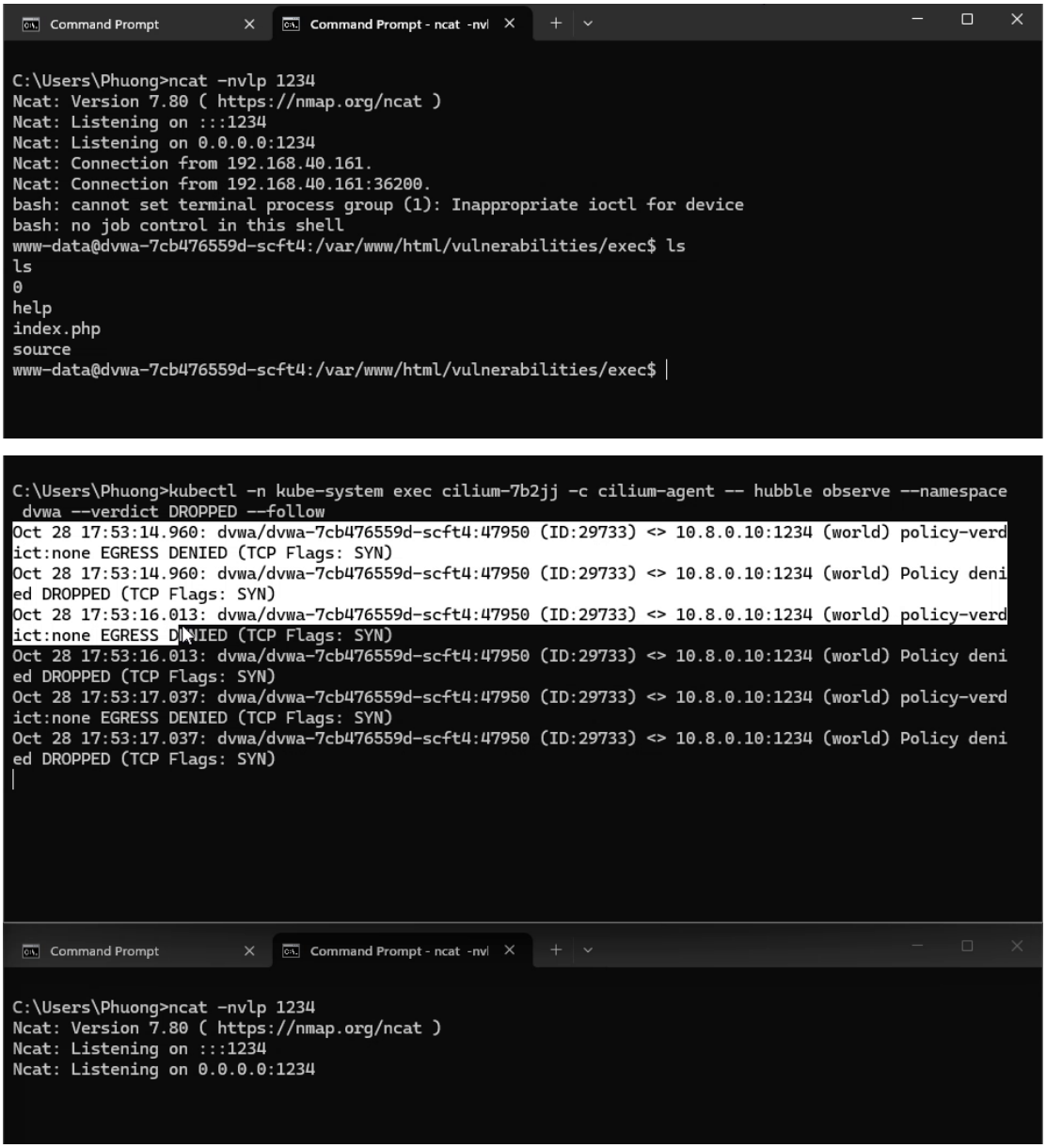
  egress:

    - toEndpoints:
        - matchLabels:
            k8s:io.kubernetes.pod.namespace: kube-system
            k8s:k8s-app: kube-dns
      toPorts:
        - ports:
            - port: "53"
              protocol: UDP
            - port: "53"
              protocol: TCP


    - toEntities:
        - kube-apiserver
      toPorts:
        - ports:
            - port: "6443"
              protocol: TCP
            - port: "443"
              protocol: TCP
```

Hình 13: Policy `dvwa-restrict-egress`: giới hạn egress của DVWA, cho phép DNS trên UDP/TCP 53 và Kubernetes API trên TCP 443/6443.

Kết quả quan sát. Sau khi giới hạn egress, Hubble ghi nhận kết nối từ `dvwa/dvwa-7cb476559d-scft4` (identity 29733), cổng nguồn 47950, tới `10.8.0.10:1234` (`world`) với kết quả `EGRESS DENIED` và `Policy denied DROPPED`. Các gói bị chặn mang cờ TCP SYN; phía máy lắng nghe không nhận được kết nối mới. Như vậy, policy đã chặn kết nối reverse shell tới máy nhận.



Hình 14: Kết quả kiểm thử reverse shell: máy lắng nghe nhận được shell từ DVWA trong phép thử đối chứng (trên); sau khi giới hạn egress, Hubble ghi nhận kết nối tới `10.8.0.10:1234` bị từ chối (dưới).

Policy egress chặn được kết nối ra ngoài trong trường hợp này, nhưng không sửa lỗ hổng Command Injection của DVWA. Kết quả mới được kiểm tra với một địa chỉ và cổng nhận reverse shell.

5.2.4 Cilium: kiểm soát HTTP API của DVWA ở lớp 7

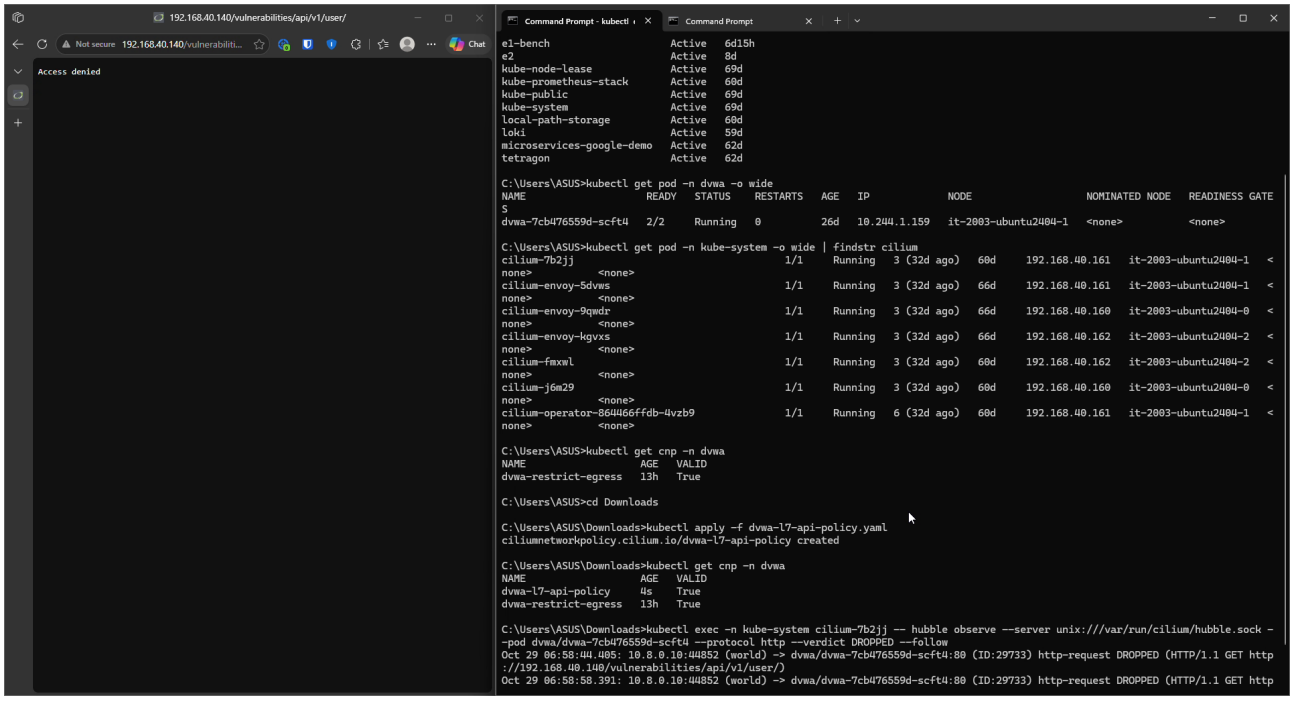
Cách kiểm tra. Ở phần L7, nhóm kiểm tra yêu cầu HTTP từ client tới API của DVWA. Trước khi áp dụng chính sách lọc HTTP, truy cập `/vulnerabilities/api/v1/user/` trả về danh sách người dùng dưới dạng JSON. Policy `dvwa-l7-api-policy` được áp dụng cho các Pod có nhãn `app=dvwa`, sử dụng `rules.http` để kiểm soát ingress trên TCP 80.

Policy khai báo một rule riêng cho các phương thức `POST`, `PUT`, `PATCH`, `DELETE`, `HEAD` và `OPTIONS`. Với GET, chỉ các đường dẫn trong danh sách cho phép mới được truy cập, gồm giao diện ứng dụng, tài nguyên hỗ trợ và API v2. Đường dẫn API v1 được kiểm thử không thuộc danh sách này. Proxy kiểm tra phương thức và đường dẫn HTTP ở L7, kết hợp với phần xử lý gói tin bằng eBPF (Mục 3.6). Kết quả này chỉ cho thấy GET bị chặn, chưa cho biết các phương thức khác có bị chặn khi gọi API v1 hay không.

```yaml
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata:
  name: dvwa-l7-api-policy
  namespace: dvwa
spec:
  endpointSelector:
    matchLabels:
      app: dvwa
  ingress:
    - fromEntities:
        - all
      toPorts:
        - ports:
            - port: "80"
              protocol: TCP
          rules:
            http:
              - method: "^(POST|PUT|PATCH|DELETE|HEAD|OPTIONS)$"
              - method: "^GET$"
                path: "^/([?].*)?$"
              - method: "^GET$"
                path: "^/(about|index|instructions|login|logout|phpinfo|security|setup)[.]php([?].*)?$"
              - method: "^GET$"
                path: "^/(dvwa|external|hackable|docs)(/.*)?$"
              - method: "^GET$"
                path: "^/(favicon[.]ico|robots[.]txt)([?].*)?$"
              - method: "^GET$"
                path: "^/vulnerabilities/(sqli_blind|captcha|exec|brute|xss_s|weak_id)(/.*)?$"
              - method: "^GET$"
                path: "^/vulnerabilities/(open_redirect|csp|javascript|xss_r|bac|authbypass)(/.*)?$"
              - method: "^GET$"
                path: "^/vulnerabilities/(cryptography|fi|upload|xss_d|sqli|csrf)(/.*)?$"
              - method: "^GET$"
                path: "^/vulnerabilities/api(/|/index[.]php)?([?].*)?$"
              - method: "^GET$"
                path: "^/vulnerabilities/api/(help|public|source)(/.*)?$"
              - method: "^GET$"
                path: "^/vulnerabilities/api/v2(/.*)?$"
```

Hình 15: Policy `dvwa-l7-api-policy` kiểm soát ingress TCP 80 theo phương thức và đường dẫn HTTP. Các đường dẫn GET được phép bao gồm API v2.

Kết quả quan sát. Lệnh `kubectl get cnp -n dvwa` xác nhận `dvwa-l7-api-policy` và `dvwa-restrict-egress` đều có trạng thái `VALID=True`. Khi truy cập lại cùng URL, trình duyệt trả về `Access denied`. Hubble ghi nhận yêu cầu `HTTP/1.1 GET` từ `10.8.0.10` (`world`) tới Pod DVWA, identity 29733, trên cổng 80 với kết quả `http-request DROPPED`. Đường dẫn trong sự kiện là `/vulnerabilities/api/v1/user/`. Kết quả cho thấy yêu cầu HTTP bị chặn theo rule L7. Trong hai trường hợp trước, Cilium chặn ngay gói SYN ở L3/L4.



Hình 16: Yêu cầu GET tới API v1 của DVWA bị chặn: trình duyệt trả về `Access denied`, đồng thời Hubble ghi nhận verdict `http-request DROPPED`.

5.2.5 Tổng hợp và đánh giá kết quả Cilium

Phép thử	Kết quả ban đầu	Kết quả thực nghiệm
Frontend tới Redis/Payment	Hai kết nối TCP báo thành công	Monitor ghi `Policy denied`, kèm IP, cổng, identity của hai luồng SYN.
DVWA reverse shell	Listener nhận kết nối và shell	Hubble ghi egress bị từ chối tới `10.8.0.10:1234`.
GET API v1 của DVWA	URL trả dữ liệu JSON	Sau áp dụng policy L7, trình duyệt báo `Access denied`; Hubble ghi HTTP request bị drop.
Thêm Pod / policy map	Không có cặp bản chụp trước–sau	Chưa đo được số entry thay đổi hoặc thời gian cập nhật chính sách.

Bảng 5: Kết quả kiểm thử chính sách mạng trên cụm Cilium.

Nhận xét. Qua ba trường hợp trên, nhóm có thể dùng log Cilium để biết Pod nào đang kết nối và kết nối đó bị chặn vì lý do gì. Với lưu lượng L3/L4, log chứa địa chỉ IP, cổng, security identity và lý do loại bỏ gói tin. Với HTTP, Hubble còn ghi lại phương thức và đường dẫn được gọi. Các thông tin này giúp nhóm phân biệt kết nối bị policy chặn với kết nối thất bại do nguyên nhân khác.

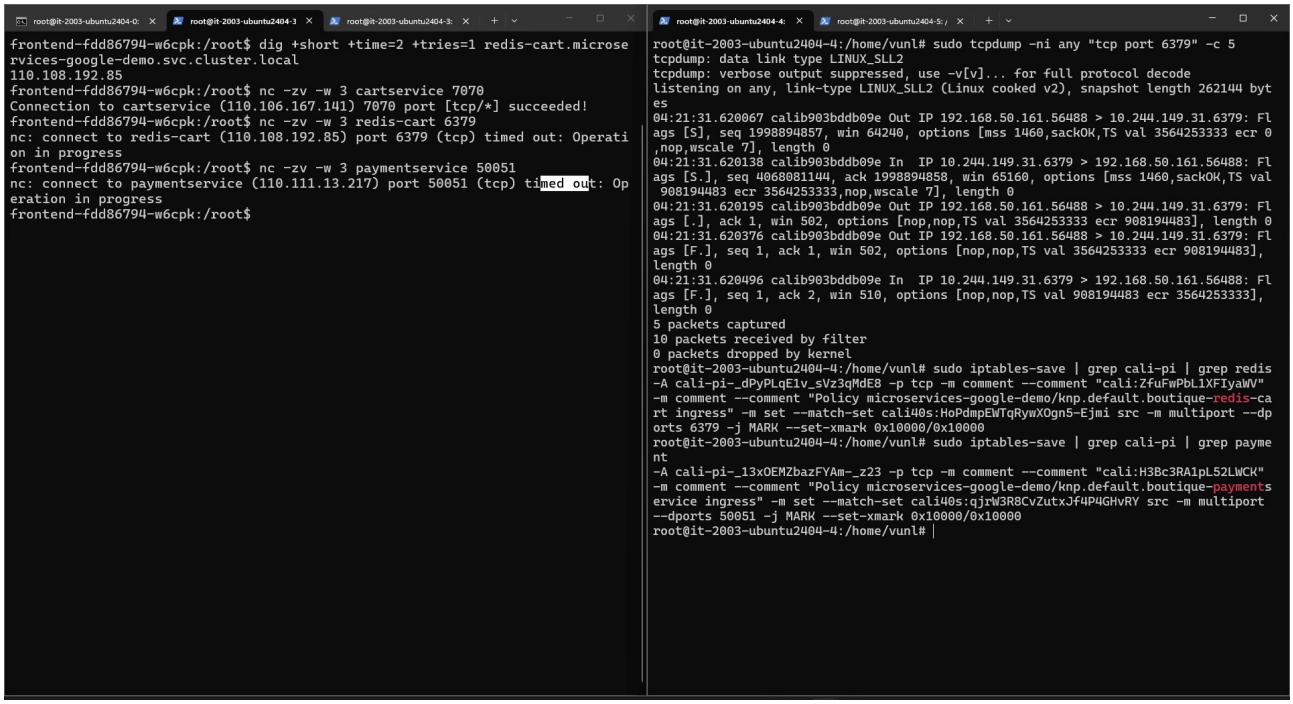
Những phần chưa kiểm tra. Nhóm mới kiểm tra hai kết nối trực tiếp từ frontend, một đích reverse shell và một yêu cầu GET tới API v1. Nhóm chưa kiểm tra đầy đủ các kết nối được cho phép, các phương thức HTTP khác và thời gian cập nhật chính sách. Nhóm chưa lưu nội dung `cilium_policy_*`, `cilium_ipcache` hoặc kết quả lệnh `cilium-dbg bpf policy get`; nên chưa biết số phần tử trong map thay đổi như thế nào khi thêm Pod. Để kiểm tra phần này, cần lưu lại endpoint, identity và nội dung map trước và sau khi tạo một Pod mới.

Cụm Calico dùng dataplane iptables như đã nêu ở Mục 4.1. Mục tiếp theo trình bày kết quả tương ứng trên cụm Calico, dùng cùng ứng dụng và cùng kiểu kịch bản kiểm thử, để có cơ sở so sánh trực tiếp với các kết quả Cilium vừa nêu.

5.2.6 Calico: giới hạn kết nối giữa các dịch vụ

Cách kiểm tra. Để đối chiếu trực tiếp với phép thử lateral movement ở Mục trước, nhóm áp dụng một bộ `NetworkPolicy` chuẩn của Kubernetes (`networking.k8s.io/v1`) cho cùng ứng dụng Online Boutique trên cụm Calico (cấu hình ở Mục 4.1: dataplane iptables, `kube_proxy_mode: iptables`). Bộ chính sách gồm một rule mặc định chặn ingress/egress và các rule cho phép theo đúng luồng nghiệp vụ, đặt tên để đối chiếu với phía Cilium – ví dụ `boutique-redis-cart` chọn Redis và cho phép ingress từ Cart trên TCP 6379, dùng `podSelector` thay cho `endpointSelector` của Cilium. Từ một container hỗ trợ kiểm tra mạng trong Pod frontend, nhóm gọi lần lượt `nc -zv cartservice 7070`, `nc -zv redis-cart 6379` và `nc -zv paymentservice 50051`.

Kết quả quan sát. Kết nối tới `cartservice` (ClusterIP `110.106.167.141`) thành công (`succeeded!`); `tcpdump` trên node chứa Pod đích xác nhận đủ bắt tay ba bước và đóng kết nối sạch (5 gói, không gói nào bị nhân loại bỏ). Kết nối tới `redis-cart` (`110.108.192.85`) và `paymentservice` (`110.111.13.217`) đều hết thời gian chờ (`timed out`). Tra lại luật trên node bằng `iptables-save | grep cali-pi`, nhóm tìm thấy hai luật mang chú thích `knp.default.boutique-redis-cart` và `knp.default.boutique-paymentservice` – tiền tố `knp.default.<tên-policy>` do Felix tự gắn khi dịch `NetworkPolicy` chuẩn, dùng để xác nhận đúng chính sách nào đã khớp, thay cho trường `status.conditions` không tồn tại ở loại tài nguyên này (Mục 3.6).

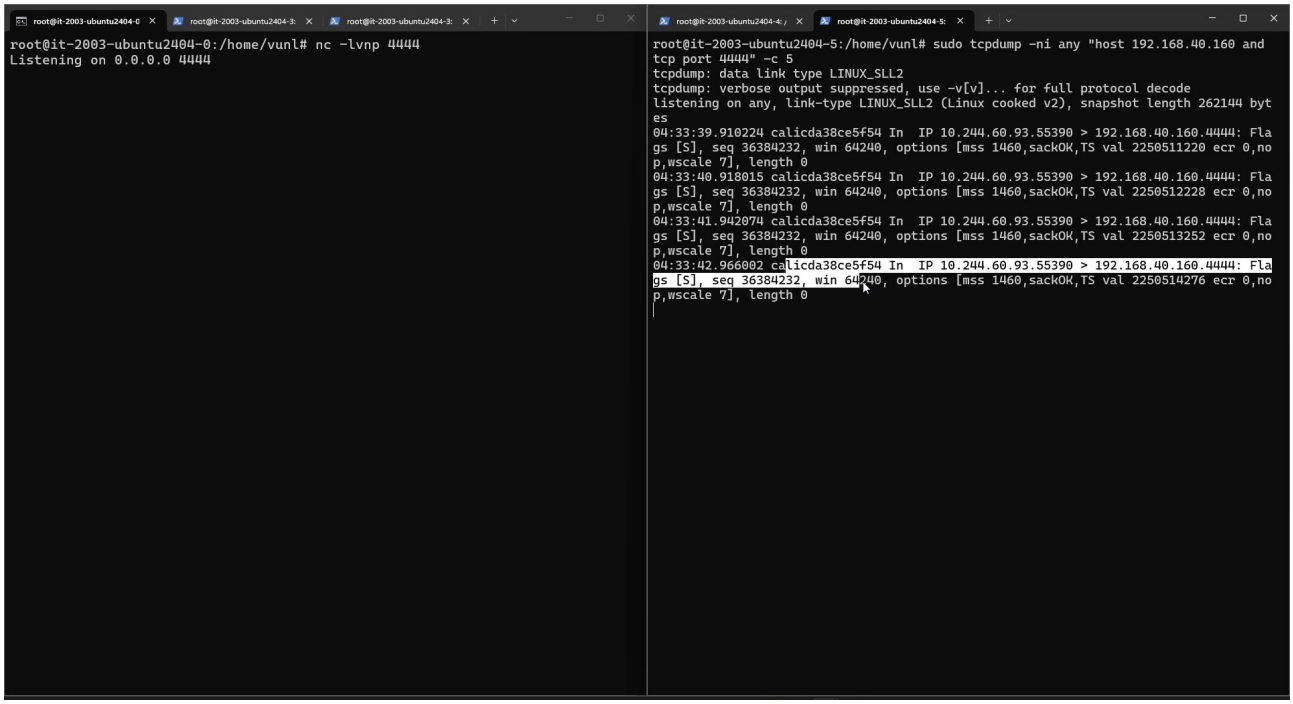


Hình 17: Cụm Calico: kết quả `nc`/`dig` từ frontend (trái) và luật iptables tương ứng, kèm tên chính sách (phải).

5.2.7 Calico: chặn kết nối reverse shell từ DVWA

Cách kiểm tra. Tương tự Mục trước về DVWA phía Cilium, nhóm áp dụng một chính sách chặn egress cho Pod DVWA trên cụm Calico. Vì `NetworkPolicy` chuẩn không có khái niệm ICMP, chính sách này dùng loại riêng của Calico (`projectcalico.org/v3`) [7], áp bằng `calicoctl` do cụm ở chế độ lưu trữ KDD (không có Calico API server riêng). Chính sách cho phép ingress TCP 80, egress ICMP (phục vụ chức năng `ping` của DVWA) và egress DNS tới `kube-dns`; mọi kết nối TCP egress khác, bao gồm cổng nhận reverse shell, không nằm trong danh sách cho phép. Nhóm gửi payload Command Injection mở kết nối ngược từ Pod DVWA tới cổng 4444 của một máy ngoài cụm, trong khi bắt gói trên node chứa Pod DVWA và theo dõi máy nghe ở phía "kẻ tấn công".

Kết quả quan sát. Máy nghe (`nc -lvnp 4444`) không ghi nhận bất kỳ kết nối nào. `tcpdump` trên node chứa Pod DVWA ghi lại bốn lần truyền lại cùng một gói SYN (cùng `seq 36384232`), cách nhau khoảng một giây, không có gói `SYN-ACK` trả lời – đúng hành vi truyền lại TCP khi gói bị loại bỏ trên đường đi. Hai bằng chứng độc lập (không có kết nối tới, không có phản hồi) cùng xác nhận egress đã bị chặn.



Hình 18: Cụm Calico: máy nghe ngoài cụm không ghi nhận kết nối (trái); `tcpdump` trên node chứa Pod DVWA ghi nhận 4 lần truyền lại SYN không có `SYN-ACK` (phải).

Giống nhận xét ở Mục Cilium, phép thử này chỉ xác nhận egress bị chặn với một địa chỉ và cổng cụ thể, không sửa lỗ hổng Command Injection của DVWA. Nhóm chưa thực hiện phép thử lọc HTTP API ở lớp 7 trên Calico, vì `NetworkPolicy` chuẩn và loại chính sách riêng của Calico đều không đọc được nội dung tầng ứng dụng (Mục 3.6) – đây là giới hạn kiến trúc của Calico bản mã nguồn mở, không phải thiếu sót khi triển khai.

5.2.8 Tổng hợp và đánh giá kết quả Calico

Phép thử	**Kết quả ban đầu**	**Kết quả thực nghiệm**
Frontend tới Cart	–	`nc` báo `succeeded`, `tcpdump` xác nhận bắt tay ba bước và đóng sạch.
Frontend tới Redis/Payment	Chưa kiểm tra trước khi áp policy	`nc` báo `timed out`; `iptables-save` cho đúng tên luật `knp.default.boutique-redis-cart` / `boutique-paymentservice`.
DVWA reverse shell	Chưa kiểm tra trước khi áp policy	Máy nghe không nhận kết nối; `tcpdump` ghi 4 lần truyền lại SYN không có `SYN-ACK`.
GET API v1 của DVWA (L7)	–	Không thực hiện được: `NetworkPolicy` chuẩn và chính sách riêng của Calico đều không đọc nội dung HTTP.
Thêm Pod / nội dung ipset	–	Chưa đo được, cùng tình trạng với phía Cilium.

Bảng 6: Kết quả kiểm thử chính sách mạng trên cụm Calico.

Nhận xét. Qua hai trường hợp trên, bằng chứng phía Calico cần kết hợp hai nguồn độc lập (`tcpdump` và `iptables-save`) thay vì một nguồn duy nhất như Hubble phía Cilium, và cần truy cập trực tiếp vào node dữ liệu qua SSH – trong khi phía Cilium chỉ cần `kubectl exec` từ máy điều khiển. Đổi lại, luật iptables mang tên chính sách cụ thể (`knp.default.<tên-policy>`), cho bằng chứng rõ ràng không kém phía Cilium đối với hai luồng này.

Những phần chưa kiểm tra. Giống phía Cilium, nhóm chưa lưu nội dung `ipset` hay log của Felix trước/sau khi thêm Pod, nên chưa thể so sánh số lượng mục hay thời gian cập nhật giữa hai dataplane – đây là khoảng

trống chung của RQ2, không riêng một giải pháp. Nhóm cũng chưa kiểm tra đầy đủ các luồng được phép trên cụm Calico (ví dụ chạy thử toàn bộ quy trình mua hàng), tương tự giới hạn đã nêu ở phía Cilium.

5.3 E3 – Giám sát hành vi truy cập tệp và kết nối mạng bằng Tetragon

Mục tiêu thí nghiệm. Trong E3, nhóm dùng Tetragon để theo dõi các tiến trình, thao tác đọc tệp và kết nối TCP trong Pod DVWA. Nhóm kiểm tra ba trường hợp: chạy lệnh qua Command Injection, theo dõi kết nối của tiến trình web và đọc các tệp ServiceAccount qua File Upload kết hợp File Inclusion. Các cấu hình chỉ ghi lại sự kiện, không chặn lệnh, đọc tệp hay kết nối mạng.

5.3.1 Cấu hình và cách theo dõi

DVWA chạy trong namespace `dvwa`, trên node `it-2003-ubuntu2404-1`, với IP Pod `10.244.1.159`. Nhóm đọc sự kiện từ Pod Tetragon `tetragon-n7zxv` trên cùng node và lọc theo tên Pod DVWA. Lệnh theo dõi dùng trong thí nghiệm là:

Listing 4: Lọc sự kiện Tetragon theo Pod DVWA.

```
kubectl -n kube-system exec tetragon-n7zxv -c tetragon -- tetra getevents -o compact --pods dvwa-7
    cb476559d-scft4
```

Với giám sát tiến trình, Tetragon đã ghi được sự kiện `process` và `exit` khi chưa có `TracingPolicy` bổ sung. Để theo dõi kết nối TCP và đọc tệp, nhóm áp dụng thêm hai policy:

- `e3-tcp-connect-monitor`: theo dõi hàm `tcp_connect`, lấy tham số đầu tiên với kiểu `sock` và dùng hành động `Post` để gửi sự kiện.
- `e3-file-access-monitor`: theo dõi hàm `security_file_permission`, lấy thông tin tệp và mặt nạ quyền truy cập, đồng thời thu thập giá trị trả về của hàm. Policy lọc tên tệp có đuôi `/token`, `/ca.crt` hoặc `/namespace` bằng `Postfix`; điều kiện `Mask` có giá trị 4 dùng để chọn thao tác đọc.

Hai cấu hình trên dùng loại `TracingPolicy`, không phải `TracingPolicyNamespaced`. Các rule không khai báo selector riêng cho Pod DVWA; bộ lọc `--pods` trong lệnh đọc sự kiện giúp nhóm chỉ xem kết quả của Pod đang kiểm tra. Đây là hai bước khác nhau: chọn sự kiện cần thu thập trong policy và lọc kết quả hiển thị khi đọc log.

5.3.2 Theo dõi tiến trình từ Command Injection

Cách kiểm tra. Nhóm dùng chức năng Command Injection của DVWA để chạy lệnh `ping`, sau đó thử thêm lệnh in chuỗi `Hello world`. Trên terminal, nhóm theo dõi các tiến trình được tạo ra trong Pod bằng `tetra getevents`.

Kết quả. Tetragon ghi lại tiến trình `/bin/sh` với tham số dòng lệnh, tiến trình con `/usr/bin/ping` và các sự kiện `exit`. Khi chạy `ping -c 4 127.0.0.1`, tiến trình ping và shell đều kết thúc với mã 0. Trong lần thử có thêm `echo`, dòng lệnh của shell chứa cả phần in chuỗi `Hello world`. Lệnh echo không xuất hiện thành một tiến trình riêng trong log; nhóm vẫn thấy được nội dung lệnh qua tham số của shell.

```
C:\Users\Phuong>kubectl get tracingpolicies
No resources found

C:\Users\Phuong>kubectl -n kube-system exec tetragon-n7zxv -c tetragon -- tetra getevents -o compact -
-pods dvwa-7cb476559d-scft4
🚀 process dvwa/dvwa-7cb476559d-scft4 /bin/sh "-c -- \"ping  -c 4 127.0.0.1\""
🚀 process dvwa/dvwa-7cb476559d-scft4 /usr/bin/ping "-c 4 127.0.0.1"
💥 exit    dvwa/dvwa-7cb476559d-scft4 /usr/bin/ping "-c 4 127.0.0.1" 0
💥 exit    dvwa/dvwa-7cb476559d-scft4 /bin/sh "-c -- \"ping  -c 4 127.0.0.1\"" 0
🚀 process dvwa/dvwa-7cb476559d-scft4 /bin/sh "-c -- \"ping  -c 4 ; echo 'Hello world'\""
🚀 process dvwa/dvwa-7cb476559d-scft4 /usr/bin/ping "-c 4"
💥 exit    dvwa/dvwa-7cb476559d-scft4 /usr/bin/ping "-c 4" 2
💥 exit    dvwa/dvwa-7cb476559d-scft4 /bin/sh "-c -- \"ping  -c 4 ; echo 'Hello world'\"" 0
```

Hình 19: Tetragon ghi lại các tiến trình và mã thoát trong Pod DVWA. Dòng lệnh của shell cho thấy lệnh ping và phần lệnh echo được chạy qua Command Injection.

Mã thoát cần được đọc theo từng tiến trình. Ở lần thử có echo, lệnh ping thiếu địa chỉ đích nên trả mã 2, trong khi shell kết thúc với mã 0. Vì vậy, mã thoát của shell không đủ để kết luận mọi lệnh bên trong đều chạy thành công.

5.3.3 Theo dõi kết nối TCP

Cấu hình. Nhóm áp dụng `e3-tcp-connect-monitor` để ghi lại các lần tiến trình gọi `tcp_connect`. Cấu hình lấy thông tin socket và gửi sự kiện lên Tetragon, không có hành động chặn kết nối.

```yaml
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: e3-tcp-connect-monitor
spec:
  kprobes:
    - call: tcp_connect
      syscall: false
      args:
        - index: 0
          type: sock
      selectors:
        - matchActions:
            - action: Post
```

Hình 20: Policy theo dõi hàm `tcp_connect`, với tham số kiểu `sock` và hành động `Post`.

Kết quả. Sau khi gửi yêu cầu tới DVWA, log có sự kiện `connect` của `/usr/sbin/apache2` từ các cổng nguồn `50972` và `50986` tới `127.0.0.1:3306`. Cùng lúc, Tetragon vẫn ghi các sự kiện process và exit của shell, ping. Nhờ đó, nhóm thấy được tiến trình nào mở kết nối, địa chỉ nguồn, địa chỉ đích và cổng được sử dụng.

```
C:\Users\Phuong>kubectl get tracingpolicies
NAME                     AGE
e3-tcp-connect-monitor   9s

C:\Users\Phuong>kubectl -n kube-system exec tetragon-n7zxv -c tetragon -- tetra getevents -o compact --pods dvwa-7cb476559d-scft4
  connect dvwa/dvwa-7cb476559d-scft4 /usr/sbin/apache2 tcp 127.0.0.1:50972 -> 127.0.0.1:3306
  connect dvwa/dvwa-7cb476559d-scft4 /usr/sbin/apache2 tcp 127.0.0.1:50986 -> 127.0.0.1:3306
  process dvwa/dvwa-7cb476559d-scft4 /bin/sh "-c -- \"ping  -c 4 127.0.0.1\""
  process dvwa/dvwa-7cb476559d-scft4 /usr/bin/ping "-c 4 127.0.0.1"
  exit    dvwa/dvwa-7cb476559d-scft4 /usr/bin/ping "-c 4 127.0.0.1" 0
  exit    dvwa/dvwa-7cb476559d-scft4 /bin/sh "-c -- \"ping  -c 4 127.0.0.1\"" 0
```

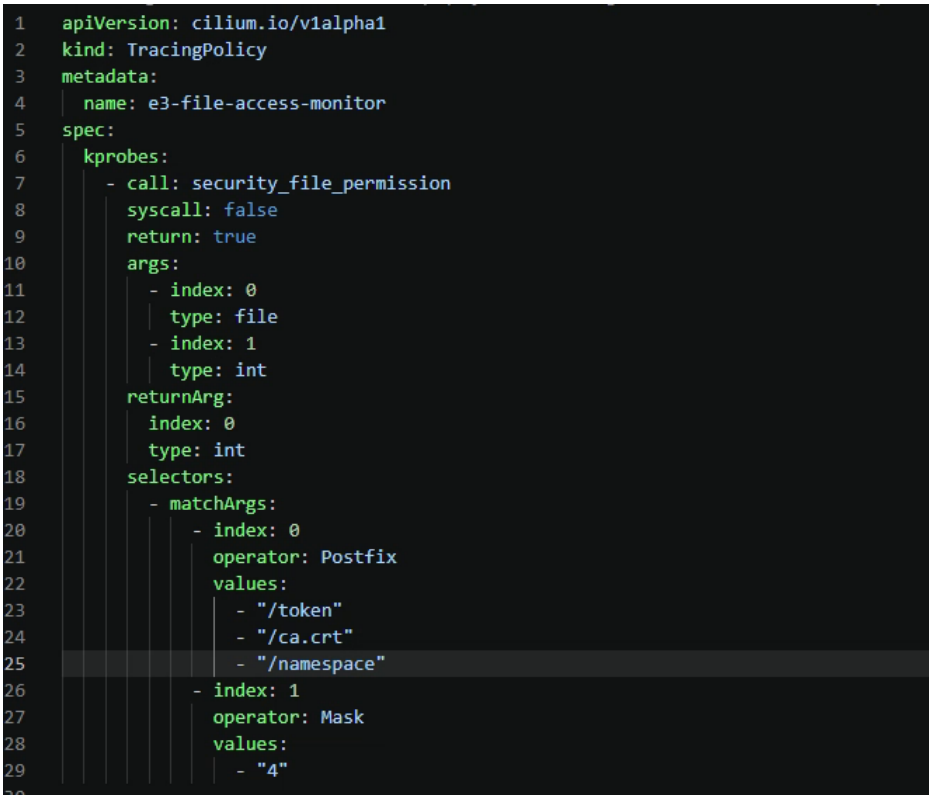
Hình 21: Sự kiện `connect` của Apache tới `127.0.0.1:3306`, cùng các sự kiện tiến trình trong Pod DVWA.

Cần phân biệt hai hoạt động trong phép thử này: ping dùng ICMP, còn các dòng connect của Apache là kết nối TCP riêng. Policy theo dõi `tcp_connect` không phải công cụ ghi lại toàn bộ gói tin TCP hay lưu lượng ping. Sự kiện connect cho biết tiến trình gọi hàm khởi tạo kết nối; nhóm chưa dùng riêng sự kiện này để kết luận dữ liệu ứng dụng đã được gửi hoặc nhận thành công.

5.3.4 Theo dõi đọc tệp ServiceAccount và gọi Kubernetes API

Cách kiểm tra. Nhóm tải tệp PHP kiểm thử `exploit.php` lên DVWA rồi gọi tệp này qua File Inclusion. Chương trình đọc thông tin ServiceAccount và gửi yêu cầu tới Kubernetes API. Trong trường hợp này, mã PHP chạy trong tiến trình web đang có sẵn, nên chỉ xem các tiến trình mới được tạo ra sẽ chưa đủ để biết chương trình đã truy cập tệp nào.

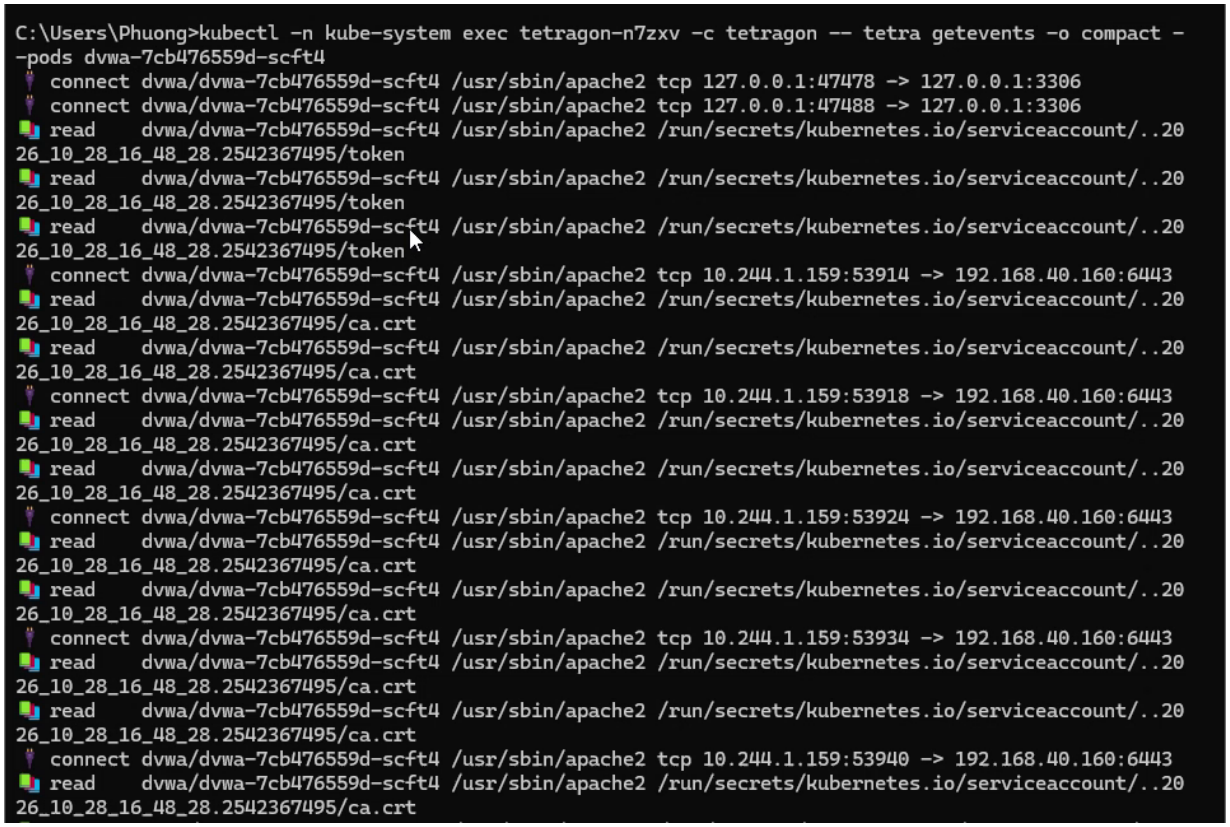
Policy `e3-file-access-monitor` được dùng để bổ sung sự kiện đọc tệp. Bộ lọc kiểm tra đuôi tên tệp thay vì toàn bộ đường dẫn. Vì vậy, rule có thể khớp cả tệp cùng tên ở vị trí khác; khi đọc kết quả, nhóm cần kiểm tra đường dẫn đầy đủ và tên tiến trình.



```yaml
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: e3-file-access-monitor
spec:
  kprobes:
    - call: security_file_permission
      syscall: false
      return: true
      args:
        - index: 0
          type: file
        - index: 1
          type: int
      returnArg:
        index: 0
        type: int
      selectors:
        - matchArgs:
            - index: 0
              operator: Postfix
              values:
                - "/token"
                - "/ca.crt"
                - "/namespace"
            - index: 1
              operator: Mask
              values:
                - "4"
```

Hình 22: Policy theo dõi đọc tệp qua hàm `security_file_permission`. Rule lọc các đuôi `/token`, `/ca.crt`, `/namespace` và mặt nạ quyền đọc có giá trị 4.

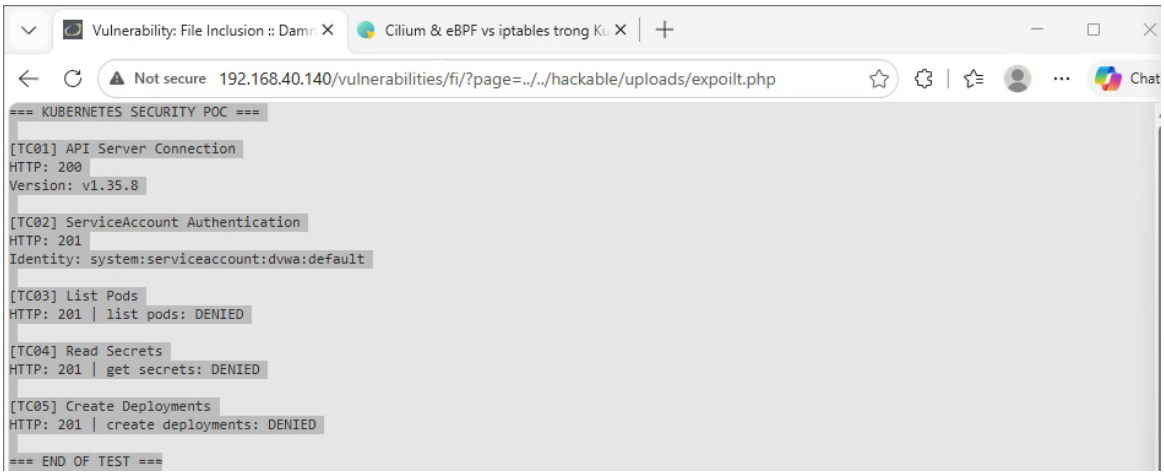
Kết quả. Sau khi áp dụng policy, Tetragon ghi các sự kiện `read` của `/usr/sbin/apache2` đối với token và `ca.crt` trong thư mục `/run/secrets/kubernetes.io/serviceaccount/`. Đường dẫn có thêm thư mục phiên bản do Kubernetes gắn vào Pod. Cùng trong kết quả theo dõi, tiến trình Apache mở các kết nối từ `10.244.1.159` tới `192.168.40.160:6443`, là địa chỉ Kubernetes API của cụm.



Hình 23: Tetragon ghi nhận Apache đọc token, tệp CA của ServiceAccount và gọi `tcp_connect` tới `192.168.40.160:6443`. Các sự kiện đều thuộc Pod DVWA.

Các sự kiện này cho thấy cùng tiến trình web có thao tác truy cập tệp ServiceAccount và mở kết nối tới API. So với log tiến trình, log đọc tệp bổ sung được đường dẫn cụ thể dù không có một lệnh đọc tệp chạy thành tiến trình mới. Tuy nhiên, sự kiện tại `security_file_permission` chỉ cho thấy thao tác kiểm tra quyền đọc; nó không ghi lại nội dung hay số byte đã đọc. Kết quả đang có cũng chưa cho thấy sự kiện đọc tệp `namespace`, dù tệp này nằm trong bộ lọc.

Kết quả ở ứng dụng. Chương trình kiểm thử trả về HTTP 200 khi gọi API lấy phiên bản và hiển thị `v1.35.8`. Phần kiểm tra danh tính hiển thị `system:serviceaccount:dvwa:default`. Các bước kiểm tra quyền có dòng `DENIED` đi cùng HTTP 201, nên nhóm chưa dùng các dòng này để kết luận quyền thực tế của ServiceAccount. Cần kiểm tra lại cách chương trình đọc nội dung phản hồi của API.



Hình 24: Kết quả chương trình kiểm thử trong DVWA: truy cập API lấy phiên bản và thông tin ServiceAccount. Các dòng kiểm tra quyền cần được kiểm tra lại cách xử lý phản hồi.

5.3.5 Nhận xét

Nội dung	Kết quả ghi được	Thông tin dùng để kiểm tra
Tiến trình	Shell, ping và các sự kiện exit	Tên Pod, đường dẫn chương trình, tham số lệnh, mã thoát.
Kết nối TCP	Apache gọi connect tới cổng 3306 và Kubernetes API cổng 6443	Tiến trình, IP nguồn/đích và cổng.
Đọc tệp	Apache truy cập token và tệp CA của ServiceAccount	Tiến trình, Pod và đường dẫn tệp.

Bảng 7: Kết quả giám sát bằng Tetragon trong E3.

Qua ba trường hợp trên, nhóm thấy mỗi loại sự kiện cung cấp một phần thông tin khác nhau. Log tiến trình cho biết lệnh nào được chạy; log kết nối cho biết tiến trình gọi tới đâu; log đọc tệp cho biết tiến trình truy cập tệp nào. Dùng cả ba giúp nhóm theo dõi được hành vi của ứng dụng khi chạy lệnh mới và khi PHP xử lý ngay trong tiến trình Apache.

Những phần chưa kiểm tra. Nhóm chưa đo chi phí CPU, bộ nhớ hoặc tỉ lệ mất sự kiện khi tải cao. Các cấu hình hiện tại chỉ giám sát, nên kết quả không thể hiện khả năng chặn truy cập tệp hay kết nối mạng của Tetragon. Để kiểm tra một yêu cầu API có được chấp nhận hay không, vẫn cần xem mã HTTP và nội dung phản hồi của ứng dụng, thay vì chỉ dựa vào dòng connect trong log.

6 Kết Luận

Trong quá trình thực hiện đề tài, nhóm đã tìm hiểu và phân tích cơ chế hoạt động của mạng trong Kubernetes, tập trung vào hai hướng tiếp cận là Netfilter/iptables và eBPF thông qua Cilium. Các nội dung được nghiên cứu bao gồm quá trình xử lý gói tin trong Linux Kernel, cơ chế định tuyến, quản lý trạng thái kết nối, thực thi NetworkPolicy và khả năng giám sát lưu lượng mạng giữa các Pod.

Đối với hướng tiếp cận truyền thống, Netfilter cung cấp các hook trong Linux Kernel để thực hiện những tác vụ như lọc gói tin, NAT và theo dõi kết nối. Công cụ iptables cho phép người quản trị thiết lập các quy tắc xử lý thông qua tables và chains, trong khi việc thực thi các quy tắc này diễn ra trong kernel. Đây là cơ chế đã được sử dụng rộng rãi trong Linux networking và nhiều thành phần mạng của Kubernetes.

Trong khi đó, Cilium tận dụng eBPF để triển khai các chương trình xử lý lưu lượng tại những hook phù hợp trong kernel. Thông qua việc kết hợp các eBPF programs và BPF Maps, Cilium có thể quản lý trạng thái kết nối, xác định Security Identity và thực thi NetworkPolicy mà không cần truy vấn Kubernetes API Server cho từng gói tin. Ngoài ra, Cilium còn hỗ trợ chính sách mạng ở Layer 3, Layer 4 và Layer 7, đồng thời cung cấp khả năng quan sát lưu lượng thông qua Hubble.

Từ những nội dung đã phân tích, nhóm tổng hợp một số đặc điểm chính của hai hướng tiếp cận trong Bảng 8.

Nhìn chung, Cilium với nền tảng eBPF mang lại nhiều lợi thế đáng chú ý, đặc biệt ở khả năng tối ưu quá trình xử lý gói tin, mở rộng chính sách bảo mật và quan sát hoạt động mạng. Việc sử dụng Security Identity cũng giúp chính sách không phụ thuộc hoàn toàn vào địa chỉ IP của từng Pod. Đây là một điểm phù hợp với đặc điểm của Kubernetes, nơi các Pod thường xuyên được tạo mới, thay thế hoặc di chuyển giữa các node.

Tuy nhiên, Cilium không phải lúc nào cũng là lựa chọn phù hợp cho mọi hệ thống. Việc eBPF tương tác sâu với Linux Kernel khiến quá trình triển khai và xử lý sự cố có thể phức tạp hơn, đặc biệt khi cần kiểm tra các chương trình eBPF, BPF Maps, routing và cơ chế thực thi NetworkPolicy. Bên cạnh đó, một số tính năng còn phụ thuộc vào phiên bản kernel và cấu hình Cilium đang sử dụng.

Vì vậy, việc lựa chọn CNI dựa trên eBPF hay giải pháp mạng sử dụng Netfilter/iptables cần được xem xét dựa trên quy mô cụm Kubernetes, mức độ phức tạp của hệ thống, yêu cầu bảo mật, khả năng giám sát và kinh nghiệm của đội ngũ vận hành. Với các hệ thống có nhiều microservices và yêu cầu kiểm soát lưu lượng chi tiết, Cilium là một lựa chọn đáng cân nhắc. Ngược lại, những giải pháp dựa trên Netfilter vẫn có thể đáp ứng tốt các hệ thống có yêu cầu đơn giản hơn hoặc đã có quy trình vận hành ổn định.

Về hướng phát triển, nhóm có thể tiếp tục mở rộng nghiên cứu bằng cách xây dựng các kịch bản thử nghiệm trên những cụm Kubernetes có quy mô khác nhau. Các thử nghiệm có thể tập trung vào độ trễ, thông lượng, mức sử dụng CPU và bộ nhớ khi số lượng Pod, Service và NetworkPolicy tăng lên. Ngoài ra, việc khảo sát sâu hơn về eBPF Host Routing, Cilium Hubble và cơ chế thực thi NetworkPolicy ở Layer 7 cũng là những hướng

Bảng 8: Tổng hợp sự khác biệt giữa Netfilter/iptables và Cilium eBPF

Tiêu chí	**Netfilter/iptables**	**Cilium eBPF**
Cơ chế xử lý	Sử dụng các Netfilter hooks và hệ thống tables, chains, rules để xử lý gói tin trong Linux Kernel.	Sử dụng các chương trình eBPF được gắn tại những hook phù hợp để xử lý lưu lượng, tra cứu BPF Maps và thực hiện các hành động như forward, drop hoặc redirect.
Quản lý trạng thái kết nối	Sử dụng cơ chế connection tracking của Netfilter để theo dõi các kết nối mạng.	Có thể sử dụng BPF Conntrack Maps để theo dõi trạng thái các network flows trong eBPF datapath.
NetworkPolicy	Thường dựa trên các quy tắc lọc lưu lượng ở Layer 3 và Layer 4. Khả năng cụ thể phụ thuộc vào CNI triển khai.	Hỗ trợ NetworkPolicy dựa trên Security Identity ở Layer 3, Layer 4 và có thể mở rộng lên Layer 7 thông qua proxy.
Khả năng mở rộng	Việc quản lý số lượng lớn các quy tắc có thể làm tăng độ phức tạp và chi phí cập nhật rule.	Sử dụng BPF Maps và các cơ chế tra cứu trong datapath, giúp tối ưu việc xử lý chính sách trong nhiều kịch bản.
Hiệu năng	Phụ thuộc vào số lượng quy tắc, cấu hình Netfilter, trạng thái kết nối và đặc điểm lưu lượng.	Có khả năng tối ưu đường đi gói tin bằng eBPF, nhưng mức cải thiện thực tế phụ thuộc vào cấu hình và workload.
Observability	Có thể sử dụng các công cụ như conntrack, tcpdump và hệ thống logging để theo dõi lưu lượng.	Tích hợp Hubble để quan sát network flows, Security Identity và các quyết định thực thi NetworkPolicy.
Triển khai và vận hành	Dựa trên những thành phần đã quen thuộc trong Linux networking, với hệ sinh thái công cụ tương đối phổ biến.	Đòi hỏi hiểu biết về eBPF, BPF Maps, cấu hình datapath và khả năng tương thích với Linux Kernel.

có thể tiếp tục nghiên cứu. Qua đó, nhóm có thể kiểm chứng cụ thể hơn những khác biệt đã phân tích về mặt kiến trúc và cơ chế hoạt động.

Tài liệu

[1] The Kubernetes Authors. Cluster networking. Kubernetes Documentation, https://kubernetes.io/docs/concepts/cluster-administration/networking/, 2025.

[2] James Strong and Vallery Lancey. *Networking and Kubernetes: A Layered Approach*. O'Reilly Media, Sebastopol, CA, 2021.

[3] Michal Rostecki and Swaminathan Vasudevan. Replacing iptables with eBPF in Kubernetes with Cilium. FOSDEM 2020, https://archive.fosdem.org/2020/schedule/event/replacing_iptables_with_ebpf/, 2020. SUSE.

[4] Cilium / Tetragon Authors. Overview: Tetragon – eBPF-based security observability and runtime enforcement. https://tetragon.io/docs/overview/, 2025.

[5] Cilium Authors. eBPF maps. Cilium Documentation, https://docs.cilium.io/en/latest/network/ebpf/maps/, 2025.

[6] Micheal Garner. Container networking deep dive part 8: Network policy enforcement compared, iptables vs eBPF. Blog kỹ thuật, n.d. TODO: bổ sung URL.

[7] Tigera / Project Calico Authors. Calico network policy. Calico Documentation, https://docs.tigera.io/calico/latest/network-policy/get-started/calico-network-policy, 2025.